



Informe Final de Trabajo Profesional de Ingeniería en Informática

Oxidized Neural Orchestra

Sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce.

Tutor: Ing. Ricardo Alfredo Veiga
rveiga@fi.uba.ar

Co-Tutor y Orientador: Dr. Ing. José Ignacio Alvarez-Hamelin
ihameli@fi.uba.ar

Estudiantes

Alejo Ordoñez (*Padrón 108397*)
alordonez@fi.uba.ar

Lorenzo Minervino (*Padrón 107863*)
lminervino@fi.uba.ar

Marcos Bianchi Fernández (*Padrón 108921*)
mbianchif@fi.uba.ar

Facultad de Ingeniería, Universidad de Buenos Aires

Índice

Aval del director	4
Agradecimientos	5
Resumen	6
Palabras clave	7
Abstract	8
Keywords	9
1. Introducción	10
2. Estado del arte	11
3. Oxidized Neural Orchestra (ONO)	13
3.1 Problema detectado y/o faltante	14
3.2 Solución implementada	14
3.2.1 Componentes generales de la arquitectura	15
3.2.2 Decisiones de diseño	15
4. Simulaciones y Resultados	17
4.1 Resultados	17
4.1.1 Parameter Server sincrónico con un solo servidor	18
4.1.2 Ring All-Reduce	19
4.1.3 Parameter Server sincrónico con N_S servers	19
4.1.4 Parameter Server asincrónico	20
4.1.5 Strategy-Switch	20
4.1.6 Resultados obtenidos	21
4.2 Metodología aplicada	22
4.2.1 Gestión y roles	23
4.2.2 Proceso de desarrollo	23
4.2.3 Seguimiento, tickets y gestión del alcance	24
4.2.4 Automatización	24
4.2.5 Criterios de aceptación	24

4.2.6	Artefactos e Indicadores	25
4.2.7	Riesgos Iniciales	25
4.3	Herramientas externas utilizadas	27
4.3.1	Bibliotecas de Rust	27
4.3.2	Bibliotecas de Python	28
4.4	Experimentación y validación	29
4.4.1	Pruebas automatizadas	29
4.4.2	Entorno de experimentación y reproducibilidad	29
4.4.3	Estudio derivado: impacto de las épocas offline	29
4.4.4	Síntesis de la validación	35
5.	Conclusiones y Trabajo Futuro	36
5.1	Impactos económico, sociales y ambientales del proyecto	37
5.2	Trabajos futuros	38
5.3	Conclusiones	40
6.	Aportes individuales	41
7.	Referencias	43
8.	Anexos	45
8.1	Anexo A: Repositorio del proyecto	45
8.2	Anexo B: Documentación técnica	45
8.3	Anexo C: Reproducibilidad de los experimentos	46
8.4	Cronograma de las actividades realizadas	47
8.4.1	Fases efectivamente recorridas	47
8.4.2	Contraste entre lo planificado y lo ejecutado	47
8.4.3	Carga horaria efectiva	48
8.5	Riesgos materializados y lecciones aprendidas	50
8.5.1	Riesgos previstos que se materializaron	50
8.5.2	Riesgos no previstos	50
8.5.3	Desvíos de alcance	50
8.5.4	Lecciones aprendidas	51

Aval del director

En Buenos Aires a [día] de [mes] del año [año] se reúnen el Ing. Ricardo Alfredo Veiga con los estudiantes de la carrera de Ingeniería en Informática: el Sr. Alejo Ordoñez (DNI 44480134), el Sr. Lorenzo Minervino (DNI 42720539) y el Sr. Marcos Bianchi Fernández (DNI 43874791). El objetivo de la reunión es revisar el Informe Final del Trabajo Profesional de Ingeniería en Informática.

Luego de haber leído el informe del Trabajo Profesional “Oxidized Neural Orchestra: sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce”, el Director, Ing. Ricardo Alfredo Veiga, declara que el mismo se corresponde con el trabajo realizado a lo largo del proyecto y avala el documento presentado como Informe Final, tanto en su completitud como en la claridad de redacción.

Firma y aclaración del director:

Ing. Ricardo Alfredo Veiga _____

Firmas y aclaraciones de los estudiantes:

Alejo Ordoñez _____

Lorenzo Minervino _____

Marcos Bianchi Fernández _____

Agradecimientos

A nuestro tutor, el Ing. Ricardo A. Veiga, por acompañar el trabajo de punta a punta y por sostener la exigencia sobre la forma del documento y sobre el rigor de lo que se afirma en él.

A nuestro co-tutor y orientador, el Dr. Ing. José Ignacio Alvarez-Hamelin, y al CoNexDat, Grupo de Redes Complejas y Comunicación de Datos de la Facultad de Ingeniería, por el marco en el que se desarrolló la Práctica Profesional Supervisada y por orientar las decisiones de diseño del sistema distribuido.

A nuestras familias y amigos, que bancaron los dos cuatrimestres que llevó este trabajo.

Resumen

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución; los algoritmos que se utilizan hacen uso intensivo de CPU y GPU, y requieren de grandes capacidades de memoria para almacenar tanto los datos de entrenamiento como el modelo en sí. Junto con la revolución de la aplicación de la inteligencia artificial, minimizar ese tiempo de entrenamiento se ha vuelto un tema central, y la estrategia más escalable para lograrlo es su ejecución distribuida.

Distribuir el entrenamiento, sin embargo, no es trivial: la estrategia de distribución elegida condiciona tanto el tiempo total de ejecución como la convergencia de la optimización. En este trabajo se implementó *Oxidized Neural Orchestra* (O.N.O.), un sistema distribuido de entrenamiento en Rust que incluye una biblioteca de redes neuronales propia, construida directamente sobre la biblioteca de arreglos numéricos *ndarray* y sin recurrir a ningún framework de aprendizaje profundo existente. Sobre esa base se implementaron los tres algoritmos que sirven de referencia en el área, *Parameter Server*, *All-Reduce* en anillo y *Strategy-Switch*, junto con dos interfaces de uso: una interfaz interactiva de terminal y una biblioteca de Python construida sobre una interfaz de funciones externas.

El sistema resultante es parametrizable y permitió comparar las estrategias entre sí bajo criterios controlados. Los estudios experimentales realizados sobre MNIST muestran que, los algoritmos sincrónicos convergen con buena exactitud a costa de un mayor tiempo de entrenamiento, que *Parameter Server asincrónico* es más rápido pero a costa de exactitud y que *Strategy-Switch* logra combinar las ventajas de ambos mundos exitosamente; y que postergar la sincronización entre nodos mediante epochs offline degrada la exactitud final de forma monótona sin aportar, en ese entorno, la ganancia de tiempo que la motiva. La elección de la estrategia en un despliegue real depende también de la heterogeneidad del hardware de los nodos y de la relación entre el tamaño del modelo y el ancho de banda de la red.

Palabras clave

Aprendizaje profundo, sistemas distribuidos, entrenamiento distribuido, paralelismo de datos, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, Rust.

Abstract

Training deep learning models requires large computational resources and long execution times; the algorithms involved make intensive use of the CPU and the GPU, and demand large memory capacities in order to store both the training data and the model itself. Together with the revolution in the application of artificial intelligence, minimizing that training time has become a central topic, and the most scalable strategy to achieve it is distributed execution.

Distributing the training, however, is not trivial: the chosen distribution strategy conditions both the total execution time and the convergence of the optimization. This work presents *Oxidized Neural Orchestra* (O.N.O.), a distributed training system written in Rust that includes a purpose-built neural network library, developed directly on top of the numerical array library *ndarray* and without relying on any existing deep learning framework. On that foundation, the three algorithms that serve as a reference in the area were implemented, *Parameter Server*, ring *All-Reduce* and *Strategy-Switch*, together with two user interfaces: an interactive terminal user interface and a Python library built on a foreign function interface.

The resulting system is parameterizable and made it possible to compare the strategies against each other under controlled fairness criteria. The experimental studies carried out on MNIST show that synchronous algorithms converge with good accuracy with a performance tradeoff, that asynchronous *Parameter Server* is faster but does not provide good convergence accuracy results, and that Strategy-Switch combines the advantages of both paradigms successfully; and that delaying synchronization between nodes through offline epochs degrades final accuracy monotonically without providing, in that environment, the time gain that motivates it. Choosing a strategy for a real deployment depends further on the heterogeneity of the node hardware and on the relation between model size and network bandwidth.

Keywords

Deep learning, distributed systems, distributed training, data parallelism, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, Rust.

Capítulo 1

Introducción

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución, y la estrategia más escalable para reducirlos es su ejecución distribuida. Sin embargo, distribuir el entrenamiento no es trivial: introduce desafíos en el diseño de la estrategia de distribución y puede degradar la convergencia de la optimización si no se realiza con cuidado. Este trabajo se propuso estudiar en profundidad las principales estrategias de entrenamiento distribuido y, a partir de dicho análisis, implementar un sistema distribuido de entrenamiento de modelos de aprendizaje profundo, con su propia biblioteca de redes neuronales, que sirviera como base para la comparación de esas estrategias y como punto de partida para el desarrollo de posibles mejoras.

El resultado es *Oxidized Neural Orchestra* (en adelante O.N.O.), un sistema distribuido escrito íntegramente en Rust que implementa tres algoritmos de entrenamiento distribuido, *Parameter Server*, *All-Reduce* en anillo y *Strategy-Switch*, sobre una infraestructura de comunicación y un motor de redes neuronales propios. Sobre ese sistema se realizaron los estudios experimentales que constituyen la validación del trabajo.

Conviene señalar desde el comienzo una particularidad de este informe. A diferencia de un trabajo cuyo objetivo es resolver un problema de negocio, aquí el producto es un *sistema de experimentación*: el valor no está solo en que el sistema funcione, sino en la evidencia que permite producir.

El presente documento se organiza de la siguiente manera. En el *Estado del arte* se relevan los avances relacionados al entrenamiento distribuido y se presentan los enfoques fundamentales que sirven de base al trabajo. El capítulo *Oxidized Neural Orchestra* reúne el *Problema detectado y/o faltante*, donde se precisa la dificultad central que se aborda, y la *Solución implementada*, que describe en detalle el sistema construido, su arquitectura y las decisiones de diseño que la explican. El capítulo *Simulaciones y Resultados* agrupa los *Resultados* esperados de cada algoritmo y su contraste con los obtenidos con O.N.O., la *Metodología aplicada* con el proceso de desarrollo efectivamente seguido, las *Herramientas externas utilizadas* y el criterio con que fueron elegidas, y la *Experimentación y validación*, que presenta uno de los dos estudios comparativos que se desprendieron de este trabajo. El capítulo *Conclusiones y Trabajo Futuro* cierra el cuerpo del informe con los *Impactos económico, social y ambiental*, los *Trabajos futuros* y las *Conclusiones*. Finalmente se incluyen los *Aportes individuales*, las *Referencias* y los *Anexos*, donde se recogen también el *Cronograma de las actividades realizadas* y los *Riesgos materializados y lecciones aprendidas*.

Capítulo 2

Estado del arte

El entrenamiento distribuido surge como una extensión de las técnicas de paralelización en *HPC* (High Performance Computing). Desde los primeros enfoques de paralelismo de datos en frameworks como TensorFlow (TensorFlow Developers, 2021) y PyTorch (Paszke et al., 2019) hasta sus implementaciones más recientes optimizadas para arquitecturas heterogéneas, la evolución de estos métodos refleja la tensión constante entre escalabilidad y coherencia de los parámetros del modelo. Los relevamientos de Ben-Nun y Hoefler (Ben-Nun & Hoefler, 2019) y de Dehghani y Yazdanparast (Dehghani & Yazdanparast, 2023) sistematizan esa tensión y sirvieron de mapa conceptual para este trabajo.

El equilibrio entre convergencia y eficiencia de comunicación ha motivado una amplia línea de investigación. En la actualidad, existen dos enfoques fundamentales que sirven como base para el desarrollo de nuevas técnicas: Parameter Server (M. Li et al., 2014) y All-Reduce (Z. Li, Davis, & Jarvis, 2017).

En el enfoque de Parameter Server, el modelo se divide en un conjunto de parámetros centralizados (no necesariamente en un único nodo), a los cuales los nodos trabajadores envían sus gradientes y desde los cuales reciben las actualizaciones, que surgen de la agregación de estas contribuciones. Este esquema admite tanto una coordinación sincrónica, en la que el servidor espera los gradientes de todos los trabajadores antes de actualizar el modelo, como una asincrónica, en la que aplica cada gradiente apenas llega; esta última mejora la escalabilidad y reduce el tiempo de entrenamiento, aunque a costa de la coherencia entre copias del modelo. El caso extremo de esa relajación es *Hogwild!* (Niu, Recht, Ré, & Wright, 2011), que directamente abraza las *race conditions* sobre los parámetros compartidos y demuestra que, bajo hipótesis de esparsidad, la convergencia se preserva.

En contraste, en All-Reduce todos los nodos intercambian gradientes de manera directa y avanzan de forma sincrónica: en cada paso todos aplican la misma actualización, lo que preserva la coherencia entre réplicas. La formulación en anillo de Patarasuk y Yuan (Patarasuk & Yuan, 2009) es óptima en ancho de banda, ya que el volumen que comunica cada nodo resulta esencialmente independiente de la cantidad de participantes, y es la que popularizó Horovod (Sergeev & Del Balso, 2018) en el ámbito del aprendizaje profundo. Trabajos posteriores como BytePS (Jiang et al., 2020) muestran que, en clústeres heterogéneos, la elección entre ambas familias deja de ser evidente.

Uno de los claros casos de combinación de estos algoritmos es Strategy-Switch (Provatas, Chalas,

Konstantinou, & Koziris, 2025), que inicia el entrenamiento de forma sincrónica sobre All-Reduce y, guiado por una regla empírica, continúa con Parameter Server asincrónico una vez que el modelo en entrenamiento se estabiliza; logrando así combinar la precisión del régimen sincrónico de *All-Reduce* con la reducción de tiempo del régimen asincrónico de *Parameter Server*.

Una línea transversal a los tres enfoques es la reducción del volumen de comunicación. Aji y Heafield (Aji & Heafield, 2017) proponen transmitir únicamente los componentes de mayor magnitud del gradiente y acumular localmente el residuo, y Lin et al. (Lin, Han, Mao, Wang, & Dally, 2018) extienden la idea con compensaciones que preservan la convergencia a tasas de compresión muy altas. Una alternativa complementaria es reducir la precisión numérica de los mensajes sin alterar la precisión del cómputo. Ambas técnicas fueron implementadas en este trabajo.

Finalmente, en el terreno del aprendizaje federado, McMahan et al. (McMahan, Moore, Ramage, Hampson, & Arcas, 2017) formalizan la idea de dejar que cada nodo entrene de forma autónoma durante varias épocas antes de sincronizar, y caracterizan el fenómeno de divergencia entre réplicas que esto induce, conocido como *client drift*. Ese mecanismo, que en O.N.O. se expone bajo el nombre de *offline epochs*, es el objeto de uno de los estudios experimentales de este informe.

Capítulo 3

Oxidized Neural Orchestra (ONO)

Este capítulo presenta el sistema construido. Primero se precisa el problema concreto que motivó el trabajo, y luego se describe la solución implementada: su arquitectura, sus componentes y las decisiones de diseño que la explican.

3.1 Problema detectado y/o faltante

El entrenamiento de un modelo de aprendizaje profundo es costoso en dos dimensiones distintas: el tiempo y la memoria. Sobre conjuntos de datos grandes, un entrenamiento completo puede extenderse durante días sobre una única máquina, y tanto el conjunto de datos como el modelo y sus estados intermedios pueden exceder la memoria disponible en un solo dispositivo (Ben-Nun & Hoeffer, 2019). A eso se suma una restricción práctica que define el escenario habitual: quien necesita entrenar estos modelos rara vez dispone de un equipo de gran porte, pero es frecuente que tenga varias máquinas de capacidad modesta. Alcanzar una capacidad de cómputo dada sumando el hardware que ya se tiene resulta bastante más accesible que adquirir un solo equipo que la iguale, y esa es la razón por la que se adopta el entrenamiento distribuido: reducir el tiempo repartiendo el cómputo entre varios nodos, y superar la restricción de memoria repartiendo los datos o los parámetros.

Sobre ese problema general se recorta la dificultad concreta que motivó este trabajo: **comparar las estrategias de distribución entre sí de forma controlada es difícil**. La literatura las describe y reporta resultados sobre ellas, pero las implementaciones de referencia disponibles, como las de TensorFlow (TensorFlow Developers, 2021) y PyTorch (Paszke et al., 2019), están optimizadas para escenarios de producción y arrastran un trabajo de ingeniería que no es uniforme entre estrategias. Detrás de sus capas de abstracción quedan ocultas exactamente las decisiones que se querría poder variar:

1. El régimen de sincronización entre nodos.
2. La ubicación del optimizador dentro del ciclo de entrenamiento.
3. El esquema de particionado de los parámetros.
4. La codificación de los mensajes que se intercambian.

Al comparar dos estrategias sobre dos implementaciones distintas, la diferencia medida mezcla el efecto del algoritmo con el de la calidad de su implementación, sin que exista forma de separar ambos efectos.

Se detectó entonces la ausencia de una base común, parametrizable y de código abierto, sobre la cual las distintas estrategias de distribución compartan la misma capa de comunicación, el mismo motor de redes neuronales y el mismo plano de control; de modo que la única variable entre dos ejecuciones sea la estrategia bajo estudio. Sin esa base, cualquier comparación entre algoritmos, y en particular la evaluación de mejoras propuestas sobre ellos, queda expuesta a un sesgo que no se puede cuantificar. Construir esa base, e instrumentarla para que produzca mediciones, es el problema concreto que este trabajo resuelve.

3.2 Solución implementada

Se construyó Oxidized Neural Orchestra (O.N.O.), un sistema distribuido de entrenamiento de modelos de aprendizaje profundo. Comprende una biblioteca de redes neuronales, una capa de comunicación con protocolo propio, un plano de control que coordina la ejecución, tres algoritmos distribuidos de entrenamiento y dos interfaces de uso.

Esta sección describe la arquitectura resultante. El énfasis está puesto en las decisiones de diseño y en las razones que las motivaron, por encima de los detalles de implementación: en un trabajo cuyo

objetivo era construir una base de comparación, son esas decisiones las que determinan si la base sirve para lo que fue construida.

3.2.1 Componentes generales de la arquitectura

1. **Capa de Comunicación (comms):** Abstrae la infraestructura de red mediante un protocolo propio optimizado. Separa el plano de control (comandos en formato JSON) del plano de datos (tensores y gradientes mediante reinterpretación de memoria sin copias).
2. **Motor de Redes Neuronales (machine_learning):** Provee la biblioteca de aprendizaje profundo (capas, funciones de activación, pérdidas, entrenadores y modelos), con inicializaciones de Xavier-Glorot (Glorot & Bengio, 2010) y Kaiming-He (He, Zhang, Ren, & Sun, 2015) y optimizadores que incluyen el descenso por gradiente con momento (Sutskever, Martens, Dahl, & Hinton, 2013) y Adam (Kingma & Ba, 2014). Su diseño organiza el modelo como un buffer de memoria plano sobre el cual operan las capas, facilitando la partición y transmisión de parámetros sin serializaciones complejas.
3. **Plano de Control y Orquestación (orchestrator):** Administra el ciclo de vida del entrenamiento. Realiza mediciones de latencia en la red para seleccionar topologías óptimas, coordina la ejecución orientada a eventos y gestiona criterios de detención temprana (Prechelt, 1998) de los entrenamientos realizados.
4. **Nodos de Cómputo (node):** Arquitectura agnóstica de rol donde todos los nodos ejecutan el mismo binario base (*node*). Según la especificación asignada dinámicamente por el orquestador, un nodo opera como *Worker* o como *Parameter Server*.
5. **Estrategias de Sincronización y Distribución:** Soporta tres esquemas de distribución integrados:
 - **Parameter Server:** Sincronización multi-servidor con particionado de capas.
 - **Ring All-Reduce:** Algoritmo descentralizado con comunicación lógica de anillo entre los nodos.
 - **Strategy-Switch:** Conmutación dinámica de All-Reduce a Parameter Server asincrónico cuando la pérdida se estabiliza. Sobre una ventana de las últimas seis pérdidas registradas se promedia la variación relativa entre pérdidas consecutivas, y el cambio de estrategia se dispara apenas ese promedio cae por debajo de un umbral fijo (0,01), siguiendo el criterio de Provatas et al. (Provatas et al., 2025).
6. **Interfaces de Usuario:** Exposición del sistema mediante una interfaz gráfica de terminal interactiva (*orchestui*) y una librería de bindings para Python vía FFI (*orchestra-py*).

3.2.2 Decisiones de diseño

Nodo agnóstico de rol. Todos los nodos ejecutan el mismo binario y el rol de cada uno es una especificación que el orquestador asigna en tiempo de ejecución. Cada conexión se identifica a sí misma en el handshake inicial, un intercambio **Connect/Accept** con su identificador y su rol declarado, de modo que el otro extremo construye el handle tipado correspondiente sin necesidad de conocer de antemano quién se está conectando. La razón de fondo es Strategy-Switch: conmutar de All-Reduce a Parameter Server en medio de un entrenamiento exige promover trabajadores a servidores sin redespargar el clúster, y eso solo es posible si el rol no queda fijado en el despliegue.

La misma propiedad deja abierta, como extensión, la participación de un mismo proceso en más de un entrenamiento.

Modelo como buffer plano. El modelo no es una estructura de objetos con sus pesos adentro: es un único buffer contiguo de memoria sobre el que las capas operan mediante vistas, y el objeto modelo conserva solo la metadata que describe cómo interpretarlo. La consecuencia buscada es que particionar el modelo entre servidores o transmitirlo por la red no exige copiar ni serializar los pesos; basta delimitar rangos del buffer.

Separación del plano de control y el plano de datos. Los comandos de coordinación viajan en JSON, legible y depurable; los tensores y gradientes viajan por reinterpretación de memoria sin copias. La división responde a que ambos planos tienen necesidades opuestas: el de control es poco frecuente y conviene poder inspeccionarlo; el de datos concentra el volumen y cada copia intermedia cuesta.

Particionado respetando límites de capa. Cuando Parameter Server fragmenta el modelo entre varios servidores, el reparto se realiza por capas enteras. Así, cada gradiente producido por una capa viaja por completo a un único servidor, sin partir tensores al enviar ni recomponerlos al recibir.

Selección de topología por latencia medida. El orquestador mide la latencia entre los nodos antes de entrenar y con esas mediciones decide el orden del anillo y la ubicación de los servidores. La topología resulta de la red real y no de la configuración, lo que evita que una asignación arbitraria introduzca diferencias de tiempo ajenas al algoritmo.

Ubicación del optimizador: gradiente plano en los workers, estado en el servidor. Los workers siempre computan localmente un descenso por gradiente sin estado, y son los servidores de parámetros los que aplican el optimizador con estado (momento, Adam) sobre el gradiente agregado. La razón es doble: evita transmitir el estado del optimizador por la red en cada ronda, y en All-Reduce directamente no sería aplicable, porque los buffers internos de un optimizador con estado quedarían con valores incoherentes al momento de reducir los gradientes de distintos workers.

Actualización de parámetros sin bloqueos, detrás de un trait polimórfico. El servidor de parámetros admite dos regímenes de actualización intercambiables en tiempo de configuración: uno con lock por fragmento (`BlockingStore`) y uno que directamente asume las *race conditions* al escribir sobre los parámetros compartidos, siguiendo a Niu et al. (Niu et al., 2011) (`WildStore`). Ambos implementan el mismo trait `Store`, de modo que la elección entre corrección estricta y velocidad queda expuesta como un parámetro (`store: blocking|wild`) y no como una bifurcación de código.

Interfaz de Python como crate separado, no como feature-flag. La primera aproximación anotaba los tipos existentes con atributos condicionales (`#[cfg_attr(feature="python-ffi", pyclass)]`) para mantener una única fuente de verdad. Se descartó por límites duros del compilador: `#[pyclass]` no admite tipos genéricos, algunas variantes de enum no tienen una forma compatible, y ciertos tipos de error hubieran requerido implementar `Clone`. `orchestra-py` se construyó entonces como un crate aparte que envuelve al `orchestrator`, manteniendo este último en Rust puro y liberando el GIL con `py.allow_threads` durante las llamadas bloqueantes.

Estas decisiones sostienen, en conjunto, la propiedad que da sentido al sistema como base de comparación: que entre dos corridas la única variable sea la estrategia de distribución.

Capítulo 4

Simulaciones y Resultados

4.1 Resultados

El objetivo principal de los algoritmos que se implementan en este trabajo es el de reducir el tiempo de entrenamiento de un modelo de deep learning, T_1 , que está dado por la suma de T_{ser} y T_{par} , los tiempos de procesamiento de las porciones de trabajo serializado y paralelizable, respectivamente. Dado que $T_{\text{ser}} \ll T_{\text{par}}$, pues, para los algoritmos que se presentan, el trabajo serializado consiste en compartir el dataset y la metadata del modelo entre los nodos, y que además T_{ser} no cambia entre los algoritmos, porque en todos ellos necesita compartirse esa información; podemos despreciar T_{ser} y hablar de T_{par} como T_1 en el contexto de la comparativa.

En un mundo perfecto, donde el costo de sincronización de los nodos es despreciable, el tiempo de entrenamiento es inversamente proporcional a la cantidad de nodos P :

$$T_P = \frac{T_1}{P}.$$

Ahora bien, el mundo no es perfecto y el costo de sincronización no es despreciable; más aún, veremos más adelante que en algunos casos puede incluso crecer con la cantidad de máquinas que deben sincronizarse.

A continuación, se realiza un análisis de los tiempos de entrenamiento esperados para cada algoritmo y se los compara con los que fueron obtenidos con O.N.O.

El tiempo total de entrenamiento distribuido está dado por la combinación del tiempo de ejecución local y el delay de sincronización y de inicialización

$$T_{\text{total}} = N_{\text{epochs}} * \left(\frac{T_{\text{epoch}}}{N_W} + d_{\text{sync}} \right) + d_{\text{inicialización}}.$$

En un entorno de computación distribuida, una pasada por el dataset en cada epoch se divide entre la cantidad de workers que las computan.

Sin perder generalidad y conservando dinamismo, se opta por mostrar los resultados contra el dataset MNIST (LeCun, Bottou, Bengio, & Haffner, 1998). Como veremos a continuación, un factor clave en

el delay de sincronización que introducen los algoritmos de entrenamiento distribuido es el tamaño del modelo S con respecto al bandwidth R de la red, es decir, el delay de transmisión d_{trans} de la red. Para poder simular un ambiente multi computadora en una sola máquina con 16 cores, se usó Docker para limitar la cantidad de CPUs asignados a cada nodo y la herramienta Pumba para simular bandwidth limitado en la comunicación de red.

Dado que el tamaño de los modelos que resultan útiles para aprender MNIST no es lo suficientemente grande para obtener diferencias observables entre los algoritmos, para tener una relación S/R buena para evaluación, se configuró $R = 10$ Mbps subrealistamente bajo, de manera tal de evitar usar modelos innecesariamente grandes para MNIST o cambiar el problema a uno que así los requiera. El modelo que se entrenó es LeNet-5, que tiene 61706 parámetros. Como los parámetros y los gradientes se comparten en la red como números flotantes de 16 bits, el tamaño del modelo a los ojos de la red es de casi 1 Mb. El delay de transmisión es entonces aproximadamente $d_{\text{trans}} = 1$ segundo. Se utilizó el optimizador Adam (Kingma & Ba, 2014) con learning rate 0.01 y batch size 64; por 48 epochs.

Por último, el tiempo total de entrenamiento en una sola máquina, sin hacer uso de ningún algoritmo distribuido, es $T_1 = 600$ segundos.

Cuadro 4.1: Configuración de los benchmarks. Parameter Server siempre se ejecuta con un único servidor.

Parámetro	Valores
Dataset	MNIST completo (LeCun et al., 1998)
Modelo	LeNet-5
Función de pérdida	Entropía cruzada
Optimizador	Adam ($\alpha = 0,01$; $\beta_1 = 0,9$; $\beta_2 = 0,999$; $\epsilon = 10^{-8}$)
Tamaño de mini-batch	64
Epochs	48
N_W	2, 4, 6, 8, 12, 16

4.1.1 Parameter Server sincrónico con un solo servidor

En Parameter Server (M. Li et al., 2014) con un solo server, el delay que introduce la sincronización del sistema está dado por el tiempo de comunicación de la ida de los parámetros del server hacia los workers, y de la vuelta de los gradientes de los workers hacia el server.

Suponiendo un mismo bandwidth R para todos los nodos, delays de encolado, propagación y procesamiento despreciables; y un modelo de tamaño S ($S = N_{\text{params}} \times 2$ bytes, dado que los parámetros se serializan como flotantes de 16 bits); el tiempo que tarda la ida de los parámetros es el de su envío en N_W mensajes unicast $N_W \frac{S}{R}$.

Suponiendo un ambiente homogéneo, en el que el tiempo de procesamiento de los workers es similar, todos comienzan a contestar con los respectivos gradientes en simultáneo. Sin embargo, el server solo tiene una tarjeta de red con la cuál recibir estos gradientes, por lo que el tiempo que tarda la vuelta de los gradientes es también $N_W \frac{S}{R}$.

El delay de sincronización de Parameter Server sincrónico con un solo servidor es entonces

$$d_{\text{sync, PS}} = 2N_W \frac{S}{R}.$$

Nótese que el delay crece con la cantidad de workers.

Cabe destacar que generalmente uno querría que el nodo que ejecuta el parameter server ejecute también un worker, dado que de otra forma se estaría desperdiciando su CPU siendo que la entidad del server es I/O bounded. En cuyo caso el delay de comunicación entre el parameter server y el worker que residen en el mismo nodo se anula, y por tanto $d_{\text{sync, PS}} = 2(N_W - 1) \frac{S}{R}$.

4.1.2 Ring All-Reduce

El delay de sincronización de Ring All-Reduce (Patarasuk & Yuan, 2009) está dado por el delay del *scatter* y por el delay del *gather*. Ambas etapas del algoritmo son de $N_W - 1$ iteraciones. En ambas, la sincronización en una iteración consiste en enviar una N_W -ésima parte del gradiente del i -ésimo al $i + 1$ -ésimo nodo. Como despreciamos los delays de encolado, propagación y procesamiento; y como las tarjetas de red son *full-duplex* (tienen canales dedicados para leer y escribir), podemos pensar que la comunicación en anillo se realiza en paralelo.

Considerando los mismos supuestos de delays de comunicación por red que usamos para el desarrollo anterior, el delay de sincronización de All-Reduce, está dado por hacer dar vuelta una N_W -ésima parte del gradiente dos veces. Esto es

$$\begin{aligned} d_{\text{sync, AR}} &= 2(N_W - 1) \frac{S/N_W}{R} \\ &= 2(1 - \frac{1}{N_W}) \frac{S}{R}. \end{aligned}$$

A diferencia de Parameter Server, el factor afectado por la cantidad de workers $\frac{1}{N_W} \rightarrow 0$ cuando $N_W \rightarrow \infty$,

$$\lim_{N_W \rightarrow \infty} d_{\text{sync, AR}} = 2 \frac{S}{R}.$$

El delay de sincronización está acotado por el doble del delay de transmisión del tamaño de los parámetros del modelo.

4.1.3 Parameter Server sincrónico con N_S servers

Como ya vimos, la mayor parte del delay de sincronización en Parameter Server viene del hecho de que todos los workers tienen que comunicarse con un único servidor, y dado que este servidor puede enviar y recibir una sola tira de parámetros/gradientes a la vez, el delay escala con N_W .

Una forma de resolver este problema es incrementar la cantidad de servidores. Cada servidor tiene una porción disjunta de los parámetros del modelo, y los workers solicitan parámetros y envían gradientes al servidor que corresponda.

En Parameter Server, por tratarse de comunicación de uno a muchos, el cuello de botella de la sincronización está del lado del servidor. Con un solo servidor ya vimos que el envío de parámetros y

la recepción de gradientes ambos tardan $N_W \frac{S}{R}$, que es el máximo entre el delay desde el punto de vista de un worker y desde el del server: $\max\left(\frac{S}{R}, N_W \frac{S}{R}\right)$. Pero con N_S servers, el delay del lado del servidor se transforma en $N_W \frac{S/N_S}{R}$. Por lo tanto

$$d_{\text{sync, Multi-PS}} = 2 \frac{N_W}{N_S} \frac{S}{R} \quad (N_S \leq N_W).$$

Notar que cuando $N_S = N_W$, $d_{\text{sync, Multi-PS}} = 2 \frac{S}{R}$, que es el mismo resultado obtenido que para All-Reduce cuando N_W crece indefinidamente.

Vale la misma aclaración que hicimos en el caso para un solo servidor con respecto a que los nodos que ejecutan un server normalmente van a estar también ejecutando un worker, por lo que los delays de transmisión se anulan en esos nodos: $d_{\text{sync, Multi-PS}} = 2 \frac{N_W - 1}{N_S} \frac{S}{R}$.

4.1.4 Parameter Server asincrónico

El delay de sincronización desarrollado en los puntos anteriores se da luego de cada ronda de epochs de *todos* los workers. Esto es, cada worker computa una epoch (o más, si `offline_epochs` está configurado) para luego sincronizar sus parámetros con el resto.

Parameter Server asincrónico libera al entrenamiento distribuido de esta restricción, dejando que el parameter server mueva los parámetros del modelo a medida que van llegando gradientes de los workers *sin necesidad de esperar al resto*. En este caso el costo de sincronización se elimina porque la sincronización simplemente desaparece, aunque aún se tiene que considerar el delay de transmisión $2 \frac{S}{R}$ de cada worker.

El problema con esta estrategia es que la convergencia empeora mucho cuando la desincronización es grande, es decir, cuando los workers mueven demasiado los pesos en la dirección sesgada que les dicta su partición de los datos; fenómeno conocido como *client drift* (McMahan et al., 2017).

4.1.5 Strategy-Switch

Strategy-Switch (Provatas et al., 2025) combina la convergencia de All-Reduce y la velocidad de ejecución de Parameter Server asincrónico. La idea es encontrar un buen mínimo local con el primer algoritmo para luego seguir bajándolo por medio del segundo.

Si consideramos que d_{sync} es el tiempo en el que los nodos permanecen sin realizar ningún cómputo, ni comunicarse directamente con otro nodo, por esperar que termine la sincronización, podemos decir entonces que luego del switch no está acotado inferiormente. En el peor de los casos, cuando todos los nodos se comunican con el parameter server en simultáneo, el delay llega a $d_{\text{sync, PS}}$.

El delay de sincronización de Strategy-Switch depende entonces de si se realizó el *switch*:

$$d_{\text{sync, SS}} = \begin{cases} 2(1 - \frac{1}{N_W}) \frac{S}{R}, & \text{antes del switch (All-Reduce)} \\ d_{\text{sync, Async PS}}, & \text{después (Parameter Server asincrónico)}. \end{cases}$$

con $0 \leq d_{\text{sync, Async PS}} \leq d_{\text{sync, PS}}$.

4.1.6 Resultados obtenidos

Se muestra primero la comparativa de resultados de los algoritmos que operan de manera sincrónica.

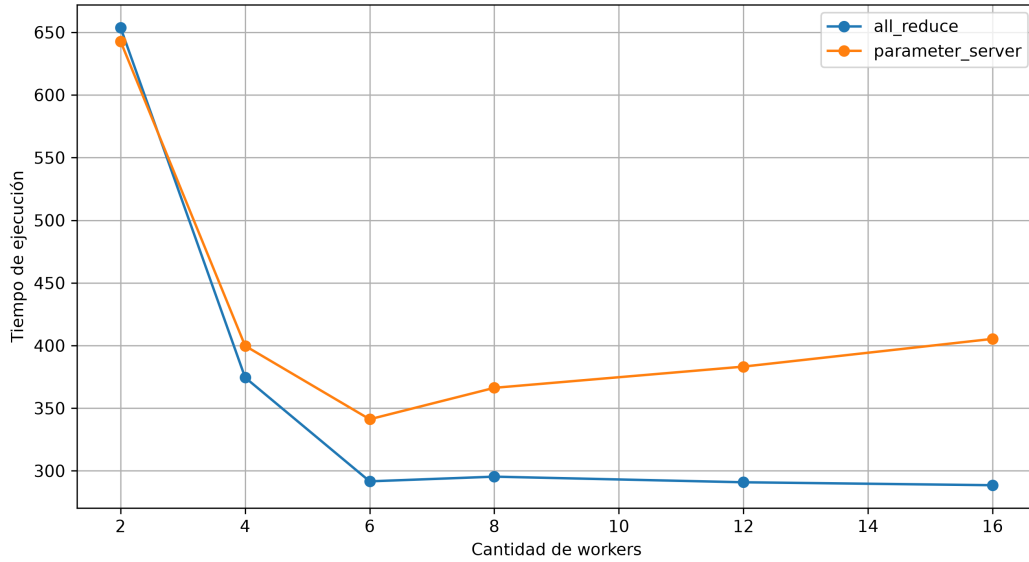


Figura 4.1: Tiempo de entrenamiento de MNIST VS. cantidad de workers para All-Reduce y Parameter Server.

Podemos observar que el delay de sincronización hace que la performance de Parameter Server empiece a empeorar a partir de $N_W = 6$. Por otro lado, la sincronización no empeora con la cantidad de workers para All-Reduce puesto que el delay está acotado, pero sí agrega una cota inferior al tiempo de ejecución.

A continuación se muestran los resultados obtenidos para Parameter Server asincrónico con un solo server y para Strategy-Switch.

Los resultados son muy prometedores y muy similares para ambos algoritmos. Sin embargo, al examinar cómo evoluciona la accuracy obtenida sobre el set de test en Parameter Server asincrónico, nos encontramos con que la acumulación de gradientes de manera descontrolada arruina por completo los resultados de convergencia.

El hecho de que cada worker empuje para su lado pesa mayormente al principio del entrenamiento, cuando los workers no se encuentran en mínimos locales acordados en conjunto.

Strategy-Switch aprovecha la velocidad de la desincronización en el momento indicado para no perder la convergencia, cuando los mínimos locales si fueron consensuados de antemano. El resultado del híbrido es una convergencia similar a la de los algoritmos sincrónicos y, como ya vimos, una velocidad de ejecución similar a la del asincrónico.

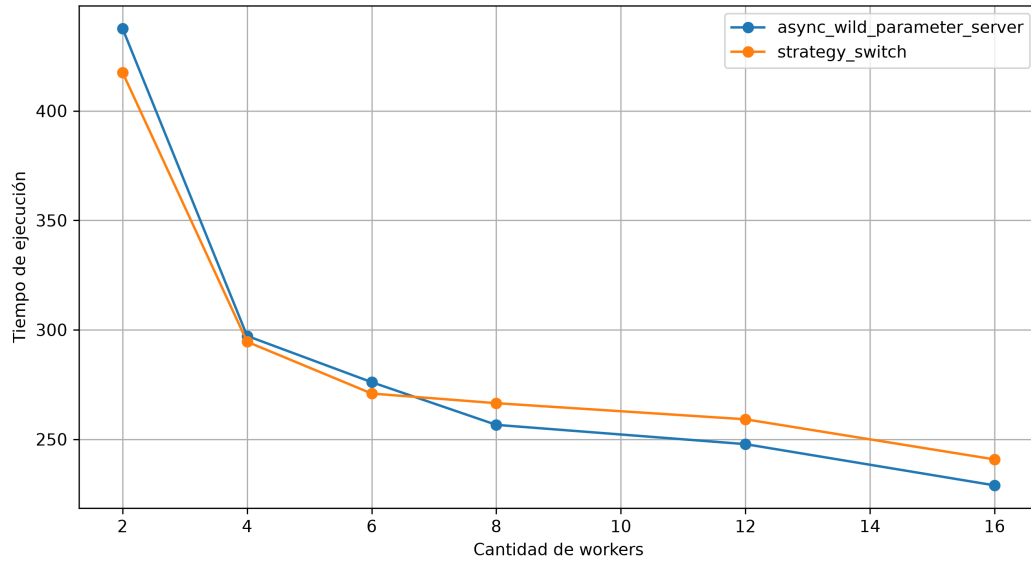


Figura 4.2: Tiempo de entrenamiento de MNIST VS. cantidad de workers para Parameter Server asincrónico y Strategy-Switch.

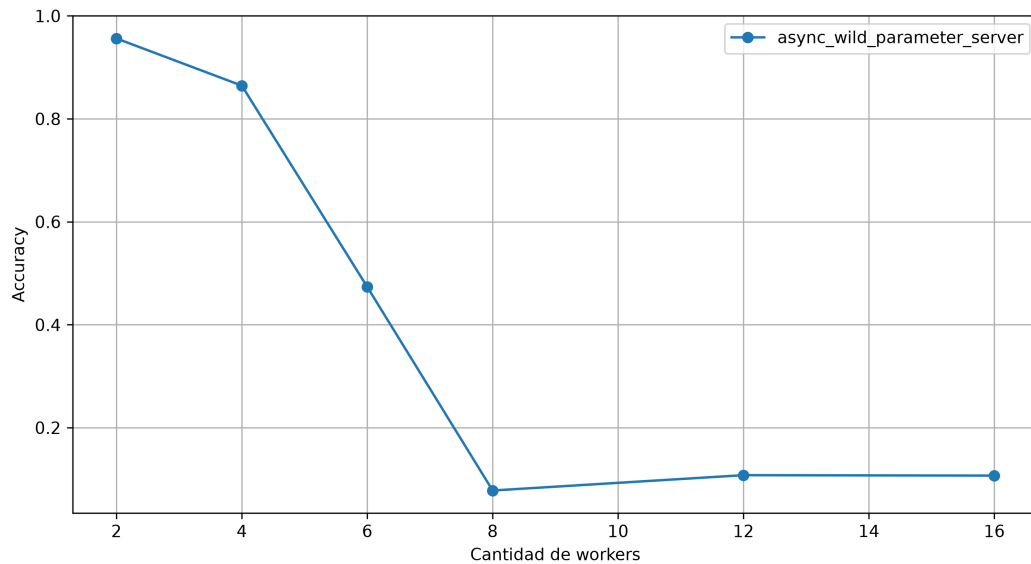


Figura 4.3: Evolución de la accuracy de MNIST VS. cantidad de workers para Parameter Server asincrónico.

4.2 Metodología aplicada

Esta sección actualiza lo planteado en la propuesta, contrastando lo previsto con lo efectivamente ejecutado.

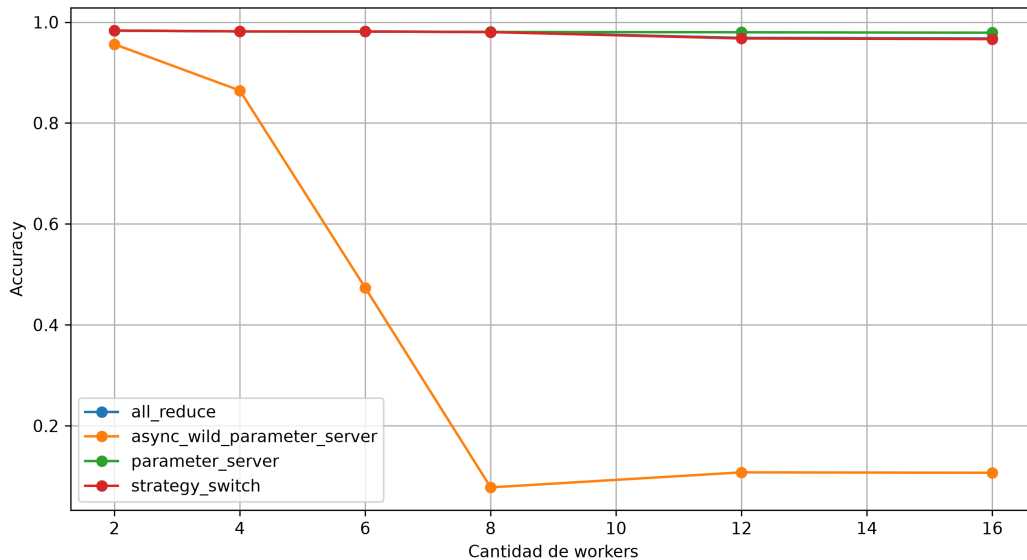


Figura 4.4: Evolución de la accuracy de MNIST VS. cantidad de workers para Parameter Server asincrónico y Strategy-Switch.

4.2.1 Gestión y roles

Se estableció un compromiso de 500 horas por estudiante a lo largo de dos cuatrimestres, lo que representa un promedio de unas 16 horas semanales por persona sobre 32 semanas. El Ing. Ricardo A. Veiga cumplió la función de tutor y el Dr. Ing. J. Ignacio Alvarez-Hamelin la de co-tutor y orientador en el campo de desempeño, en el marco del CoNexDat (Grupo de Redes Complejas y Comunicación de Datos de la Facultad de Ingeniería).

Los tres estudiantes participaron de todas las etapas de análisis, implementación y pruebas, y cada integrante concentró su foco principal en un conjunto de componentes, según se detalla en *Aportes individuales*. Esa especialización, que emergió del desarrollo y que no estaba impuesta por la propuesta, tuvo un efecto secundario relevante: la revisión cruzada de código se volvió el principal mecanismo de difusión de conocimiento entre componentes, y en varios casos las discusiones de revisión terminaron modificando decisiones de diseño y no solo detalles de implementación.

La coordinación se sostuvo en dos planos. Con los tutores se mantuvieron reuniones virtuales para informar el avance, definir prioridades y planificar próximas etapas, adecuando los encuentros según los hitos y necesidades del proyecto dentro del marco acordado. Entre los integrantes, la coordinación fue continua y de cadencia diaria, apoyada en un canal de mensajería para las decisiones rápidas y en las issues del repositorio para las que requerían registro.

4.2.2 Proceso de desarrollo

El método de desarrollo fue incremental, organizado en iteraciones con entregables intermedios. La unidad de entrega fue en forma de *pull requests*: cada funcionalidad se desarrolló en una rama propia que se integraba a la rama principal con previa revisión de los demás integrantes. A lo largo del proyecto se produjeron +1100 commits, +120 pull requests y +85 issues, entre septiembre de 2025 y

julio de 2026.

El código y todos los artefactos versionables se gestionaron con Git sobre un repositorio alojado en GitHub, y los mensajes de los commits siguen la convención de *Conventional Commits*. Los entregables intermedios que marcaron el avance fueron, en orden: la capa de comunicación funcionando de punta a punta, el primer entrenamiento distribuido completo con Parameter Server emulando una compuerta lógica XOR, el motor de redes neuronales convergiendo sobre MNIST, la interfaz de terminal mostrando un entrenamiento en vivo, All-Reduce en anillo, la librería ffi de Python, Strategy-Switch, y finalmente la suite de benchmarks con sus resultados.

4.2.3 Seguimiento, tickets y gestión del alcance

El seguimiento del proceso, la gestión de tickets y el registro de errores se realizaron con las *issues* del repositorio, categorizadas mediante etiquetas por tipo (*feature*, *bug*, *enhancement*, *documentation*) y por componente del sistema (*parameter-server*, *comms*, *worker*, *ffi*, *tui*, entre otras). Las issues se usaron además, y esto excedió lo previsto en la propuesta, como espacio de debate de diseño: varias de las decisiones de arquitectura documentadas en este informe se resolvieron enteramente en el hilo de comentarios de un issue, con enumeración de alternativas y su descarte razonado. Ese registro fue el insumo principal para reconstruir el proceso.

Los errores se clasificaron según su criticidad, separando los que frenaban el trabajo de los que podían corregirse después. La gestión del alcance se definió en dos momentos clave: al inicio, cuando se concibió el MVP, y antes del cierre, en una sesión donde se descartó deliberadamente lo que no se llegaba a desarrollar. Ambos puntos se detallan en la sección de riesgos.

4.2.4 Automatización

La construcción y las pruebas del sistema se ejecutan con las herramientas del ecosistema de Rust (`cargo test`), y los entornos de simulación se levantan de forma automatizada mediante Docker y un `Makefile` autodocumentado, lo que hace reproducible el despliegue de las distintas configuraciones de nodos.

La construcción, las pruebas, el análisis estático, la ejecución de entornos multinodo y de benchmarks completos quedaron totalmente automatizados y de ejecución desatendida: el conjunto final de 38 configuraciones se ejecutó de un solo comando durante ~6 horas sin intervención. La suite de tests se ejecuta cada vez que se commitea en un Pull Request que mergea en la rama principal mediante github actions, es obligatorio que la totalidad de las pruebas pase de forma satisfactoria para habilitar el merge.

4.2.5 Criterios de aceptación

Cada pull request se sometió a revisión de código por parte de otro integrante antes de integrarse. Una entrega se consideró aceptada cuando se cumplían de forma conjunta tres condiciones: la revisión de código fue aprobada, la totalidad de las pruebas automatizadas asociadas se ejecutaba con éxito, y la funcionalidad requerida quedaba demostrada mediante una prueba de aceptación. Para las funcionalidades que involucran coordinación entre nodos, esa prueba consistió en una ejecución

completa de entrenamiento distribuido sobre el entorno simulado, verificando que la convergencia obtenida fuera consistente con la del entrenamiento secuencial equivalente.

El criterio funcionó para las fallas visibles, pero resultó insuficiente para detectar errores de corrección distribuida silenciosos, aquellos que no provocan fallos sino que degradan la calidad del modelo. Este tipo de errores se solventó con la introducción de benchmarks y se detalla en las lecciones aprendidas.

4.2.6 Artefactos e Indicadores

Como herramienta de gestión se utilizó GitHub Projects. Cada ticket nació en el **Backlog**, pasó **En Progreso** y eventualmente, cuando el feature alcanzó un estado estable pasó a **En Revisión**, para finalmente quedar archivado en **Listo** luego de su efectiva revisión.

Algunos de los artefactos de gestión que se utilizaron durante el proceso fueron minutas de reunión para una mejor organización previa a los encuentros con los tutores y diagramas de los temas que se abarcaron, si lo ameritaban; como por ejemplo diagramas de robustez para los distintos algoritmos distribuidos que se implementaron, y diagramas de ejecución del flujo de entrenamiento agnóstico del tipo de entrenamiento mostrando la comunicación entre orquestador y nodos trabajadores.

Sobre la calidad del producto se definieron los siguientes indicadores: la proporción de cambios requeridos luego de publicar un **pull request** como listo para revisión, y el desvío entre horas planificadas y efectivamente dedicadas. Este último funcionó también como indicador de costos.

También se consideraron la proporción de pruebas exitosas sobre el total, la cantidad de advertencias del análisis estático y los resultados obtenidos de la ejecución de los benchmarks. En cuanto al análisis estático, se buscó sostenerlo sin warnings durante toda la evolución del proyecto.

4.2.7 Riesgos Iniciales

- **Complejidad de la implementación distribuida en Rust.** La coordinación entre nodos y el manejo de la concurrencia son intrínsecamente complejos y propensos a errores sutiles, como bloqueos o condiciones de carrera. Como mitigación, el desarrollo fue incremental y se apoyó en las garantías de *fearless-concurrency* del lenguaje y en una cobertura de pruebas unitarias y de integración que acompañó cada avance.
- **Disponibilidad de hardware para simulaciones representativas.** Reproducir un entorno distribuido realista requiere de múltiples máquinas, algo no siempre disponible. Para mitigarlo, las simulaciones se ejecutaron sobre contenedores Docker con límites de recursos por nodo, lo que permitió emular entornos multi computadora de forma controlada.
- **Dependencia de trabajos previos.** Parte del trabajo se apoya en algoritmos y resultados publicados, como *Strategy-Switch*, cuya interpretación e implementación pueden diferir de lo documentado. La mitigación consistió en implementar versiones de referencia de los algoritmos base y validarlas antes de construir mejoras sobre ellas.
- **Amplitud del alcance y estimación de tiempos.** El trabajo abarcó tanto el sistema base como varias líneas de optimización (comunicación, sincronización y carga en configuraciones heterogéneas), lo que dificultó la estimación del esfuerzo. Para mitigarlo, se priorizó un sistema

base funcional junto con las implementaciones de referencia, y las optimizaciones se abordaron de forma incremental según el avance.

4.3 Herramientas externas utilizadas

Rust se eligió como lenguaje del sistema principal por varios motivos:

- Control de bajo nivel sin renunciar a garantías de seguridad.
- Buen modelo de concurrencia.
- Relevancia y crecimiento en la industria.
- Familiaridad y preferencia de los integrantes.

El primer diseño de arquitectura, basado en actores sobre TCP bloqueante, se descartó tempranamente por la complejidad que agregaba la concurrencia embebida en el protocolo; migrar a un modelo *async* con la concurrencia resuelta a nivel de nodo simplificó el diseño desde ahí en adelante.

Python Se utilizó para tres propósitos:

- Interfaz de funciones externas del sistema.
- Por ser un lenguaje dominante en la industria del *Deep Learning*.
- Familiaridad con el lenguaje.

Docker Se utilizó para simular la ejecución del sistema sobre múltiples nodos y para automatizar el despliegue.

Git, GitHub, GitHub Projects y GitHub Actions Control de versiones y gestión del proceso: ramas por funcionalidad, *pull requests* con revisión obligatoria, e *issues* para el seguimiento de tickets, errores y debates de diseño. Se hizo uso de Actions para CI/CD del repositorio.

Pumba Se utilizó para simular delays de comunicación por red durante los benchmarks del sistema en entornos contenerizados de Docker. Es fácil de usar y muy parametrizable.

La totalidad de las herramientas y bibliotecas utilizadas son de código abierto, con licencias permisivas (MIT o Apache 2.0 en su mayoría). El sistema desarrollado se publica bajo licencia abierta, de modo que pueda ser retomado y extendido por terceros, según el criterio establecido en la propuesta. No se incurrió en costos de licenciamiento.

4.3.1 Bibliotecas de Rust

Estas son algunas de las bibliotecas más importantes del proyecto.

Biblioteca	Uso
<code>tokio</code>	Runtime asincrónico, TCP, tareas y canales.
<code>ndarray</code>	Operaciones sobre arreglos numéricos n-dimensionales; provee la multiplicación de matrices del motor.
<code>rayon</code>	Paralelismo de datos en el almacenamiento del servidor de parámetros.
<code>serde</code> y <code>serde_json</code>	Serialización del plano de control y de los archivos de configuración, respectivamente.
<code>bytemuck</code>	Reinterpretación de memoria sin copias entre valores numéricos y bytes.
<code>half</code>	Tipo de media precisión para la compresión de gradientes.
<code>uuid</code>	Identidad estable de nodos.
<code>safetensors</code>	Exportación del modelo entrenado.

Biblioteca	Uso
<code>log</code> y <code>tracing-subscriber</code>	Logging estructurado con filtrado por variable de entorno.
<code>ratatui</code> y <code>crossterm</code>	Interfaz interactiva de terminal.
<code>pyo3</code>	Interfaz de funciones externas con Python.
<code>anyhow</code>	Manejo de errores en la interfaz de terminal.

4.3.2 Bibliotecas de Python

Biblioteca	Uso
<code>maturin</code>	Construcción de la extensión nativa que expone el sistema a Python.
<code>numpy</code>	Lectura de los conjuntos de datos binarios en la evaluación.
<code>matplotlib</code>	Generación de todas las figuras de los experimentos.
<code>safetensors</code>	Carga de los pesos entrenados por el sistema para su evaluación.
<code>torch</code>	Línea de base de referencia.

El uso de Pytorch se restringió a benchmarks para comparar la implementación del motor de redes neuronales de O.N.O. contra una base conocida.

4.4 Experimentación y validación

La validación se apoyó en dos frentes. El primero es el de las pruebas automatizadas, que verifican que el sistema hace lo que dice hacer. El segundo, y el que da sentido al proyecto, es el de los estudios experimentales: como O.N.O. fue concebido para comparar estrategias de distribución de forma controlada, la validación última consiste en demostrar que esa base produce evidencia comparable.

4.4.1 Pruebas automatizadas

El sistema se verificó con tres tipos de pruebas. Las pruebas unitarias ejercitan cada componente de forma aislada y son las que sostienen el motor de redes neuronales: la corrección de la propagación hacia atrás no es observable a simple vista, y la única manera práctica de detectar un gradiente mal derivado es contrastarlo contra un valor calculado de forma independiente. Las pruebas de integración validan la interacción entre nodos durante una ejecución distribuida, en particular los protocolos de coordinación. Las pruebas de aceptación comprueban que cada funcionalidad requerida se comporta como fue especificada, ejecutando un entrenamiento completo sobre el entorno simulado y verificando que la convergencia obtenida sea consistente con la del entrenamiento secuencial equivalente.

Las dos primeras se escribieron con el arnés de pruebas nativo de Rust y se ejecutan mediante `cargo test`; las de aceptación se levantan sobre los entornos contenerizados descritos más abajo. Las tres son automatizadas y de ejecución desatendida, y se versionan junto con el código, de modo que cada cambio pueda validarse antes de integrarse.

4.4.2 Entorno de experimentación y reproducibilidad

El despliegue se realiza con una única imagen de Docker, coherente con el agnosticismo de rol del sistema: no hay imagen de trabajador ni imagen de servidor. Un conjunto de scripts genera la configuración del clúster, ajusta la resolución de nombres y levanta el entorno a partir de un único parámetro con la cantidad de nodos.

Todas las mediciones se realizaron sobre clústeres simulados en una única máquina física: un contenedor por nodo, red *bridge* y puertos publicados.

La suite de experimentación está escrita en Python haciendo uso de la interfaz que provee el sistema para el language, y está organizada de forma declarativa en cuatro conjuntos de ensayos que miden convergencia, velocidad de ejecución, velocidad de convergencia y escalabilidad. Los resultados se persisten de forma incremental, con repeticiones en media y desvío, y la documentación se regenera automáticamente en cada corrida. Documenta además de forma explícita sus criterios de equidad y sus fuentes de ruido: justifica el presupuesto fijo de nodos, aclara qué significa una época en cada ensayo, y explica por qué ciertas variantes se comparan por exactitud y no por velocidad.

4.4.3 Estudio derivado: impacto de las épocas offline

4.4.3.1 Pregunta e hipótesis

Permitir que los nodos operen de forma autónoma durante varias épocas posterga el intercambio de parámetros, disminuyendo la dependencia de la red y priorizando el cómputo local. Sin embargo, demorar la sincronización introduce divergencia entre los pesos de los nodos, un fenómeno conocido

como *client drift* (McMahan et al., 2017). O.N.O. expone este mecanismo como un parámetro de configuración, las épocas offline (E), lo que permite estudiarlo de forma directa.

La pregunta es: ¿Cómo impacta la variación de las épocas offline E en la velocidad de convergencia, el tiempo de comunicación y la eficacia del modelo entrenado bajo distintas topologías de red en un entorno distribuido con restricciones de red reales?

4.4.3.2 Metodología

Se ejecutaron entrenamientos sobre una misma arquitectura, conjunto de datos e hiperparámetros, variando únicamente el algoritmo distribuido, la cantidad de nodos y el valor de E . Todos los ensayos se realizaron de forma secuencial sobre la misma máquina, sumando un total de 11 horas y 15 minutos de ejecución.

Cuadro 4.4: Configuración de los ensayos. En Parameter Server, la mitad de los nodos son workers y la otra mitad servidores.

Categoría	Parámetro	Valores
Modelo y datos	Dataset	MNIST completo (LeCun et al., 1998)
	Arquitectura	LeNet-5
Optimización	Función de pérdida	Entropía cruzada
	Optimizador	Adam ($\alpha = 0,01$; $\beta_1 = 0,9$; $\beta_2 = 0,999$; $\epsilon = 10^{-8}$)
	Tamaño de mini-batch	64
Infraestructura Variables libres	Épocas máximas	48
	Serialización	Dispersa ($r = 0,5$)
	Algoritmo	All-Reduce, Parameter Server sincrónico
	Nodos	2, 4, 6, 8, 10
	Épocas offline	0, 1, 2, 3, 4

Al parametrizar $E > 0$, los nodos entrenan aislados durante múltiples ciclos y el tráfico de red se reduce de forma aproximada inversamente proporcional a E . Al alcanzar la etapa de sincronización, la consolidación se realiza por promedio directo de los gradientes parciales, de modo que bajo descenso por gradiente la actualización responde a

$$W_{t+E} = W_t - \frac{\lambda}{N} \sum_{i=1}^N G_i^{(E)}$$

Los ensayos se realizaron con la serialización dispersa que implementa el sistema, siguiendo el enfoque de Aji y Heafield (Aji & Heafield, 2017). El parámetro r acota el muestreo del gradiente que se transmite: se calcula la cardinalidad efectiva del mensaje como

$$k = \text{round}(|g| \cdot (1 - r)),$$

y ese valor k se utiliza para escoger los k elementos de mayor magnitud absoluta del tensor de gradientes local, fijando un umbral mínimo. Los elementos descartados se acumulan en un gradiente residual que se considera en el mensaje siguiente. Con $r = 0,5$, la mitad de los componentes queda fuera de cada mensaje.

El entorno de ejecución fue un procesador Intel Core i5-8350U (4 núcleos físicos, 8 hilos lógicos, 1,7 GHz base y 3,6 GHz turbo) con 8 GB de memoria DDR4, sobre Ubuntu 24.04 LTS, Docker 29.6.1, Rust 1.96.1, CPython 3.14.0 y Pumba 1.1.7.

Con Pumba se logró emular una red hogareña típica en cuestión de delay, jitter y ancho de banda.

4.4.3.3 Resultados: tiempo de ejecución

La variación de E no indujo mejoras significativas en los tiempos de ejecución: las curvas de cada configuración se solapan de forma estricta en todos los ensayos. Dado que el sistema se ejecuta en una única máquina, la supresión del costo de comunicación no alcanza a compensar la sobrecarga de cómputo, contrariamente a la expectativa inicial.

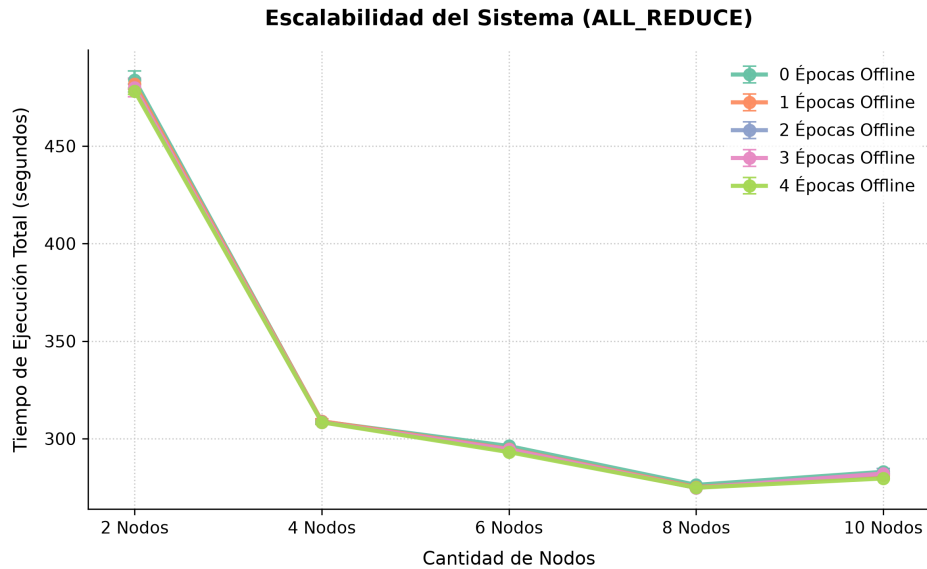


Figura 4.5: Comparación de tiempos de ejecución bajo distintas cantidades de épocas offline con All-Reduce.

La escalabilidad del rendimiento se estanca al superar los 4 nodos, y en All-Reduce la elevación a 10 nodos degrada el desempeño global por la alta contención de CPU.

En Parameter Server, en cambio, la configuración de 10 nodos (5 workers y 5 servidores) reduce el tiempo respecto de configuraciones menores. La explicación es contraintuitiva pero consistente con el entorno: mientras que en All-Reduce los procesos operan en cómputo continuo, el esquema sincrónico de Parameter Server introduce estados de inactividad obligatorios mediante barreras, y

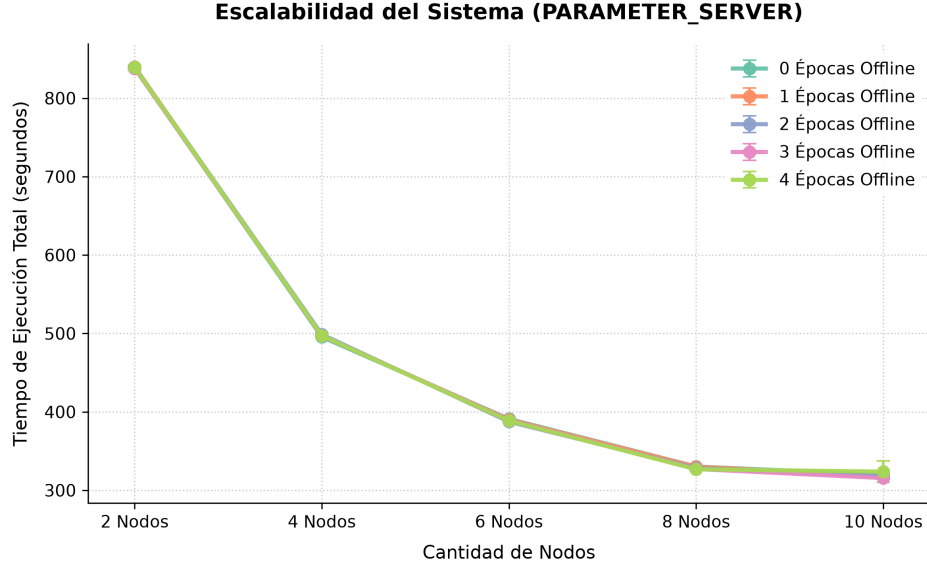


Figura 4.6: Comparación de tiempos de ejecución bajo distintas cantidades de épocas offline con Parameter Server.

esa inactividad disminuye la contención, permitiendo una alternancia más eficiente de los hilos de ejecución sobre los cuatro núcleos físicos disponibles. All-Reduce resulta, no obstante, intrínsecamente más veloz en configuraciones de pocos nodos, por su distribución balanceada de responsabilidades sin dependencias centralizadas.

4.4.3.4 Resultados: exactitud

El impacto de E se manifiesta con mayor claridad en la exactitud, por ser una métrica independiente de las limitaciones del hardware de ejecución. En ambos algoritmos, la exactitud final decrece de forma monótona al incrementar E . La introducción de una única época offline ($E = 1$), que reduce la frecuencia de sincronización global a la mitad, ya provoca un deterioro inmediato de aproximadamente un punto porcentual.

La degradación se acentúa de forma proporcional al número de nodos. Al incrementar las entidades distribuidas, las particiones locales del conjunto de datos se reducen y se vuelven potencialmente sesgadas; la falta de sincronización frecuente exacerba el impacto de esos sesgos locales, perjudicando la convergencia global.

En All-Reduce la pérdida de exactitud es más severa que en Parameter Server. Esto se explica por el diseño experimental: bajo las condiciones evaluadas, All-Reduce duplica el número de workers activos al prescindir de servidores dedicados, lo que fragmenta aún más el conjunto de datos e incrementa el aislamiento operativo. La integración tardía de parámetros fuerza entonces correcciones abruptas del gradiente global.

4.4.3.5 Resultados: inestabilidad de la pérdida

A partir del historial de entrenamiento se seleccionaron trayectorias representativas que confirman que el incremento de E introduce inestabilidad en la función de pérdida, evidenciada mediante

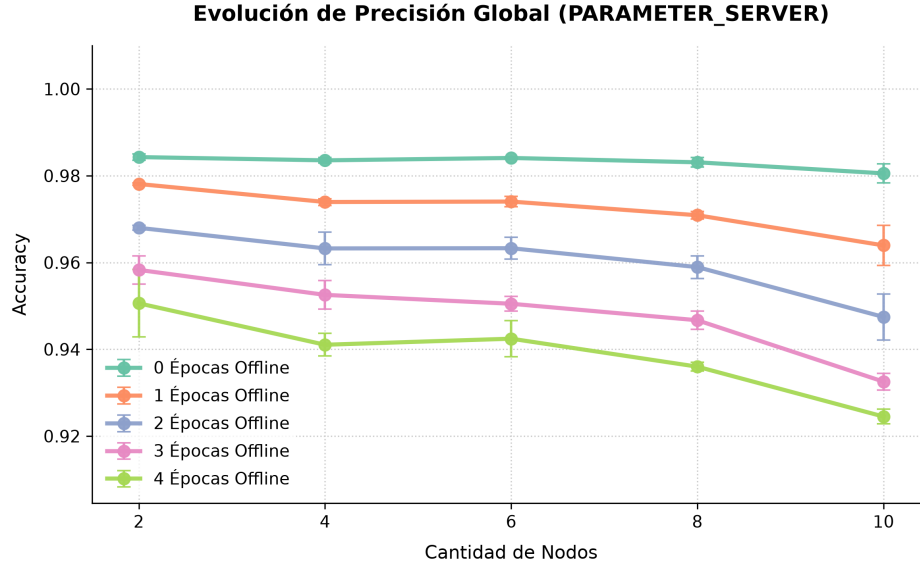


Figura 4.7: Comparación de exactitud bajo distintas cantidades de épocas offline con Parameter Server.

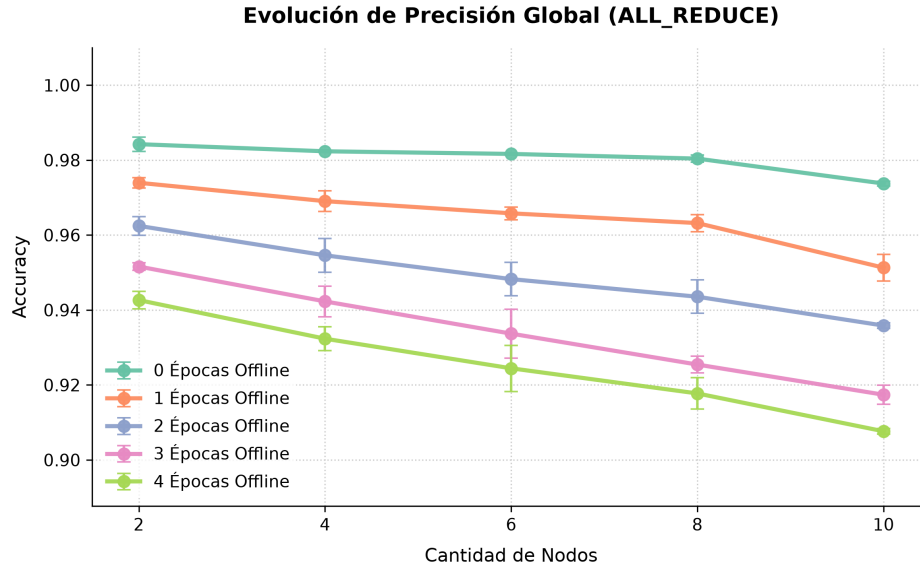


Figura 4.8: Comparación de exactitud bajo distintas cantidades de épocas offline con All-Reduce.

oscilaciones y picos proporcionales al valor de E .

La dinámica del optimizador presenta mayores dificultades en las épocas iniciales, la fase asociada a las correcciones de mayor magnitud, y luego se estabiliza al aproximarse a un mínimo local. Sin embargo, los valores de convergencia residual son sistemáticamente más altos a mayor E . El fenómeno sugiere que el desacoplamiento prolongado restringe la capacidad del optimizador para alcanzar el mínimo, induciendo un comportamiento análogo al de una tasa de aprendizaje excesivamente alta.

En All-Reduce se observa un patrón similar, aunque con oscilaciones relativamente más atenuadas,

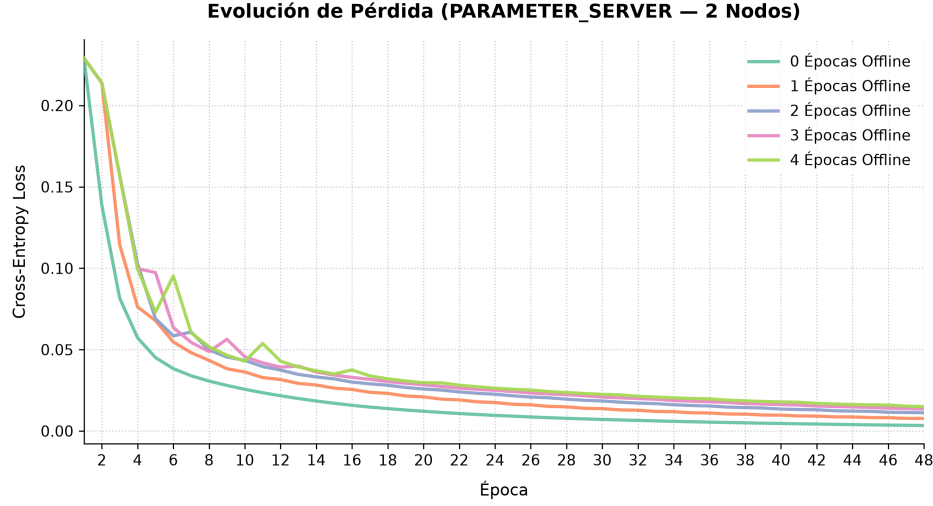


Figura 4.9: Trayectoria de pérdida con entropía cruzada exhibiendo ruido estocástico por cómputo offline en Parameter Server (2 nodos).

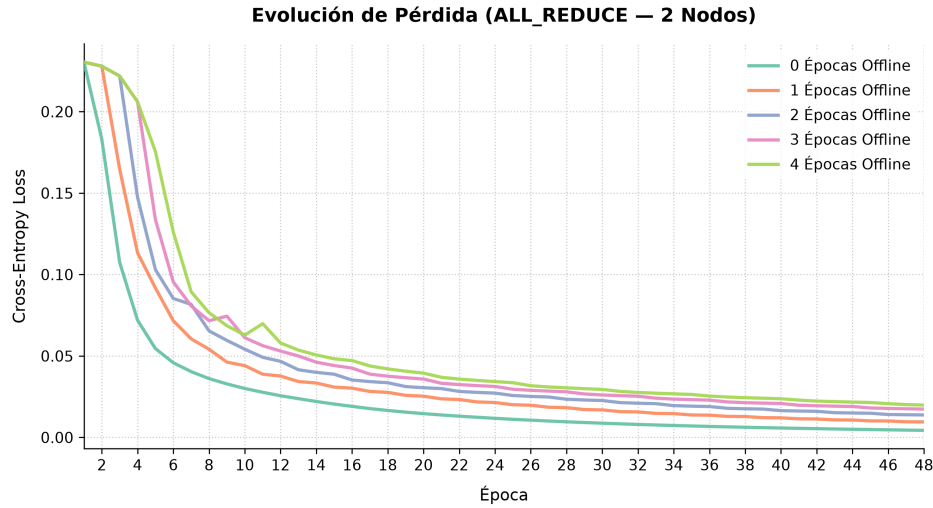


Figura 4.10: Trayectoria de pérdida con entropía cruzada exhibiendo ruido estocástico por cómputo offline en All-Reduce (2 nodos).

lo que revierte la hipótesis inicial que preveía mayor inestabilidad en el esquema descentralizado.

4.4.3.6 Discusión y limitaciones

La ventaja de introducir épocas offline es estrictamente reducir el tiempo de entrenamiento. En un entorno local simulado, reducir el costo de comunicación no compensa el costo de obtener un peor modelo: en estos experimentos no se evidenció ninguna ventaja de utilizar un E mayor a 0.

En contraposición, en un despliegue multimáquina sobre una red física, retrasar la agregación de gradientes puede resultar crítico para amortizar el tiempo de comunicación. La decisión depende de tres factores: el tamaño del modelo, la velocidad de la red y la cantidad de nodos. Con modelos

más grandes, los mensajes que contienen gradientes y parámetros son también más grandes. Para entrenamientos con mucho dato y poco modelo, el cuello de botella es el cómputo de gradientes; en cambio, para entrenamientos con poco dato y mucho modelo, donde la comunicación toma protagonismo, incrementar E puede ser útil para reducir los tiempos de ejecución.

Un resultado del estudio que merece señalarse es que la desestabilización afecta negativamente al modelo de forma predecible, lo que abre una línea concreta: evaluar estrategias de activación dinámica de las épocas offline, introduciéndolas exclusivamente en fases avanzadas del entrenamiento, cuando la optimización principal ha concluido, de modo que la exploración independiente de un worker en torno a un mínimo local pueda guiar favorablemente al resto tras la sincronización.

4.4.3.7 Conclusión del estudio

El uso de épocas offline en O.N.O. altera el balance entre comunicación y fidelidad algorítmica. En la infraestructura simulada, incrementar E perjudicó al modelo entrenado de manera predecible pero sin aportar ganancias de velocidad, debido a las limitaciones del hardware empleado. Queda planteado validar el sistema bajo configuraciones multináquina reales, comparando arquitecturas de modelos de distintos tamaños al variar E .

4.4.4 Síntesis de la validación

La observación transversal es que el sistema cumplió su propósito: validada la corrección de las estrategias por la vía de su convergencia, las diferencias de tiempo, throughput y escalabilidad admiten atribuirse al algoritmo y no a su implementación, que era exactamente el problema que este trabajo se propuso resolver.

Capítulo 5

Conclusiones y Trabajo Futuro

Este capítulo cierra el informe con los impactos económico, social y ambiental del proyecto, las líneas de trabajo que quedaron abiertas y las conclusiones generales del trabajo.

5.1 Impactos económico, sociales y ambientales del proyecto

La propuesta incluyó una evaluación preliminar de impacto. Se la revisa aquí a la luz de lo efectivamente construido y medido.

Impacto económico. Es el que este sistema comparte con cualquier plataforma de cómputo distribuido de alto rendimiento: alcanzar una capacidad de cómputo dada sale más barato sumando varias máquinas modestas que adquiriendo una sola de gama alta equivalente. El sistema aprovecha ese hecho cuando se dispone de varias máquinas, que fue la premisa de partida del trabajo. La ganancia depende de que la distribución de la carga de cómputo por nodo justifique el costo de sumarlos. El aporte económico del sistema es, entonces, permitir ubicar ese umbral antes de invertir en infraestructura. El proyecto en sí se desarrolló sobre hardware propio y con herramientas de código abierto, de modo que su costo se reduce a las 1590 horas-persona que se detallan en *Aportes individuales*.

Impacto social. El sistema se publica como base abierta y parametrizable, con implementaciones de referencia de los tres algoritmos sobre una misma infraestructura, una biblioteca de Python y una interfaz de terminal que muestra el entrenamiento en vivo. El objetivo es acercar el entrenamiento distribuido a equipos sin grandes clústeres y ofrecer un punto de partida para la investigación y la docencia. Su valor pedagógico resultó más concreto de lo previsto: al no delegar nada en un framework existente, expone la mecánica completa del entrenamiento distribuido, desde la propagación hacia atrás hasta el protocolo de red.

Impacto ambiental. El tiempo de cómputo se traduce en consumo energético, y es aquí donde la evaluación preliminar requiere la corrección más fuerte. El consumo no se midió, pero se sigue del diseño que la distribución agrega comunicación y replica el cómputo de coordinación en cada nodo: un entrenamiento sobre N nodos consume más energía agregada que el secuencial equivalente, aún cuando termine antes. Se justifica ambientalmente, entonces, cuando permite entrenar modelos que de otro modo no serían viables, o cuando se despliega sobre máquinas que de todas formas estarían encendidas. Las técnicas de reducción de tráfico implementadas apuntan a que la ganancia de tiempo se traduzca en menor uso de recursos y no solo en mayor consumo agregado. Cuantificarlo excedió el alcance del trabajo y queda como línea futura.

5.2 Trabajos futuros

El sistema quedó en un estado que admite ser retomado, y varias de las líneas que siguen tienen su diseño ya especificado en el repositorio. Se ordenan por relevancia.

Tolerancia a fallos. El trabajo planteó la ejecución del entrenamiento distribuido sobre un entorno local, donde la probabilidad de fallos se consideró muy baja, y por eso la funcionalidad no se abordó. Resulta, sin embargo, imprescindible para escalar un sistema de estas características a un entorno no local. Y aún dentro del entorno contemplado por el trabajo, donde los fallos de hardware son poco probables, quedan desatendidos los errores de usuario y los imprevistos cotidianos: una máquina que se queda sin batería, alguien que patea un cable de alimentación o que interrumpe un proceso sin querer.

El comportamiento esperado ya está especificado por entidad: el orquestador debe encolar los mensajes destinados a un par caído hasta que este se reconecte; el servidor de parámetros debe distinguir un trabajador lento de uno muerto; y el trabajador de All-Reduce debe poder esperar la reconexión de sus vecinos en el anillo. La pieza habilitante es una capa de reconexión en el transporte. Un requisito adicional ya relevado: dado que el sistema admite varias entidades sobre la misma dirección IP con puertos distintos, la reconexión necesita un servicio de resolución de nombres por el cual cada extremo anuncie, al establecerse la conexión, el puerto en el que escuchará si se cae y revive.

Resolver la actualización de parámetros libre de bloqueos de *HogWild!*, o retirarla. No se pueden tener a la vez un optimizador genérico con estado, una implementación realmente libre de bloqueos y corrección, de modo que hay dos salidas y cada una tiene su renuncia. Las operaciones atómicas dan corrección, pero solo con un optimizador sin estado, porque el ciclo de lectura-modificación-escritura de Adam o de momentum no admite ser libre de bloqueos; un bloqueo por fragmento lo resuelve, pero deja de ser la técnica del paper. Corresponde elegir de forma explícita en lugar de dejar la variante en el limbo, y cualquiera de las dos exige antes verificar empíricamente si el enfoque rinde con gradientes densos, que es el régimen en el que el sistema opera.

Variación dinámica del modelo de cómputo de la convolución. El motor ya reparte entre hilos las operaciones que se aplican capa por capa, como la actualización de parámetros y la normalización del gradiente, pero el cálculo de la convolución en sí es secuencial. Paralelizarlo llegó a implementarse y se descartó porque agregaba más sobrecarga que beneficio, con una conclusión acotada a MNIST y a modelos pequeños. También se probó utilizar convoluciones por transformadas rápidas de Fourier (Mathieu, Henaff, & LeCun, 2013) pero el *overhead* introducido por dicha estrategia también resultó demasiado para la ganancia. Con modelos, batches y kernels considerablemente mayores el balance podría invertirse, y el trabajo dejó la medición hecha como línea de base contra la cual comparar. Se deja como trabajo futuro variar las estrategias según la dimensionalidad de los datos que llegan a las capas convolucionales que los procesan y la dimensionalidad de sus kernels.

Consumo de conjuntos de datos por flujo. El sistema carga las particiones del conjunto de datos en memoria. Un mecanismo de lectura por flujo desde disco permitiría trabajar con conjuntos que no entran en la memoria de un nodo, que es uno de los dos motivos originales por los que se distribuye un entrenamiento y el que este trabajo no llegó a ejercitar.

Ampliación del motor de redes neuronales. La biblioteca cuenta con el conjunto mínimo y necesario para ejercitar los modelos con los que se propuso resolver MNIST y validar el sistema.

Queda pendiente ampliarla con más funciones de activación y extenderla a otras arquitecturas, como las redes recurrentes y las de tipo generativo.

Líneas acotadas con su alcance ya definido. En una sesión de definición de alcance previa al cierre se dejaron fuera, de forma explícita, tres extensiones que el sistema admite sin cambios estructurales: el multiplexado de sesiones sobre una misma conexión, el consumo de conjuntos de datos por flujo y la parametrización genérica del tipo de punto flotante. Cada una es autocontenida y puede retomarse por separado.

Evaluación aislada de las técnicas de reducción de tráfico. El sistema implementa codificación de gradientes en media precisión y un protocolo de gradientes dispersos con acumulación del residuo. Ambas se utilizaron en los experimentos, pero no se evaluaron de forma aislada: no se midió el compromiso entre la tasa de compresión y la degradación de la convergencia. Es un estudio autocontenido, de alcance acotado y con toda la instrumentación ya disponible.

5.3 Conclusiones

El trabajo se propuso estudiar las principales estrategias de entrenamiento distribuido de modelos de aprendizaje profundo y construir un sistema que permitiera compararlas de forma controlada. Ese objetivo se cumplió.

Sobre el producto. Se construyó O.N.O., un sistema distribuido de entrenamiento que incluye una biblioteca de redes neuronales propia, una capa de comunicación con su propio protocolo, un plano de control y dos interfaces de uso. Sobre esa base se implementaron los tres algoritmos de referencia del área, se agregaron técnicas de reducción del tráfico de red y se instrumentó todo con una suite de experimentación reproducible.

Sobre los resultados. Los estudios experimentales realizados sobre MNIST muestran que, los algoritmos sincrónicos convergen con buena exactitud a costa de un mayor tiempo de entrenamiento, que *Parameter Server asincrónico* es más rápido pero a costa de exactitud y que *Strategy-Switch* logra combinar las ventajas de ambos mundos exitosamente; y que postergar la sincronización entre nodos mediante epochs offline degrada la exactitud final de forma monótona sin aportar, en ese entorno, la ganancia de tiempo que la motiva. La elección de la estrategia en un despliegue real depende también de la heterogeneidad del hardware de los nodos y de la relación entre el tamaño del modelo y el ancho de banda de la red.

Sobre la contribución. El trabajo no innova en algoritmos: los tres implementados están publicados. La contribución es el instrumento, y responde exactamente al problema detectado en la propuesta: la ausencia de una base común sobre la cual comparar estrategias sin que la diferencia medida mezcle el efecto del algoritmo con el de su implementación. La evidencia de que esa base funciona es el conjunto de benchmarks obtenidos.

Sobre la formación. El trabajo obligó a atravesar de punta a punta tres dominios que la carrera aborda por separado. El de los sistemas distribuidos, el del aprendizaje profundo, que dejó una comprensión del entrenamiento que ningún framework habría permitido adquirir, y el de la ingeniería de software, donde la lección central fue que la instrumentación no es un anexo de medición sino una herramienta de detección, según se detalla en *Riesgos materializados y lecciones aprendidas*.

Se cierra el trabajo con un sistema funcionando, con evidencia experimental que lo respalda y con las líneas que quedaron abiertas documentadas al detalle suficiente para que otro las retome.

Capítulo 6

Aportes individuales

El desarrollo del trabajo se planteó como una responsabilidad compartida: las etapas de análisis, revisión de código y pruebas fueron llevadas adelante por los tres integrantes en todas las tareas. Sin perjuicio de ello, cada integrante concentró su foco principal en un conjunto de componentes, que se detalla a continuación y que se corresponde con lo previsto en la propuesta.

Alejo Ordoñez (Padrón 108397). Núcleo de aprendizaje automático: el modelo, las capas, los optimizadores y las funciones de pérdida. Las capas convolucionales y de *max-pooling*, incluida la reformulación de la convolución como multiplicación de matrices que produjo la mejora final de rendimiento del motor. El manejo y la distribución de los conjuntos de datos.

Lorenzo Minervino (Padrón 107863). El módulo del Worker, con participación en la implementación en anillo de *All-Reduce*. La interfaz de funciones externas en Python y la interfaz interactiva de terminal. Las optimizaciones de sincronización de las copias del modelo. Los entornos de simulación multinodo, la suite de benchmarks y el análisis experimental de resultados.

Marcos Bianchi Fernández (Padrón 108921). Los algoritmos de entrenamiento distribuido y su coordinación. El módulo de comunicaciones y el protocolo entre nodos, incluidas las técnicas de reducción de tráfico. El módulo del Servidor de parámetros. La monografía sobre el impacto de las épocas offline, que constituye el estudio experimental derivado del informe.

Cuadro 6.1: Horas dedicadas por cada integrante a cada tarea del plan de actividades.

	Alejo	Lorenzo	Marcos
Cantidad de horas insumidas	520	545	525
Leer y analizar los trabajos previos	25	20	15
Investigar la implementación distribuida de TensorFlow	0	5	0
Investigar la implementación distribuida de PyTorch	20	0	5
Desarrollar el sistema distribuido en Rust	120	70	210
Implementar Parameter Server	90	10	100
Implementar All-Reduce	10	20	20
Implementar Strategy-Switch	15	0	35

	Alejo	Lorenzo	Marcos
Optimizaciones de comunicación entre nodos	50	0	50
Optimizaciones de sincronización de las copias del modelo	20	80	0
Interfaz de funciones externas en Python e interfaz de terminal	0	100	0
Pruebas unitarias y de integración	20	60	20
Simulación sobre distintas configuraciones de nodos	30	100	20
Optimizaciones de carga en configuraciones heterogéneas (no ejecutada)	0	0	0
Análisis de resultados y gráficos	50	40	10
Documentación del código y del proceso	60	10	30
Informe final	10	30	10

Capítulo 7

Referencias

- Aji, A. F., & Heafield, K. (2017). Sparse Communication for Distributed Gradient Descent. *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, 440-445. <https://doi.org/10.18653/v1/D17-1045>
- Ben-Nun, T., & Hoeffler, T. (2019). Demystifying Parallel and Distributed Deep Learning: An In-Depth Concurrency Analysis. *ACM Computing Surveys*, 52(4). <https://doi.org/10.1145/3320060>
- Dehghani, M., & Yazdanparast, Z. (2023). From Distributed Machine to Distributed Deep Learning: A Comprehensive Survey. *arXiv preprint arXiv:2307.05232*. Recuperado de <https://arxiv.org/abs/2307.05232>
- Glorot, X., & Bengio, Y. (2010). Understanding the Difficulty of Training Deep Feedforward Neural Networks. *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics*, 249-256.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *Proceedings of the IEEE International Conference on Computer Vision*, 1026-1034.
- Jiang, Y., Zhu, Y., Lan, C., Yi, B., Cui, Y., & Guo, C. (2020). A Unified Architecture for Accelerating Distributed DNN Training in Heterogeneous GPU/CPU Clusters. *Proceedings of OSDI '20*.
- Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. *arXiv preprint arXiv:1412.6980*. Recuperado de <https://arxiv.org/abs/1412.6980>
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
- Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., ... Su, B.-Y. (2014). Scaling distributed machine learning with the parameter server. *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation*, 583-598. USA: USENIX Association.
- Li, Z., Davis, J., & Jarvis, S. (2017). An Efficient Task-based All-Reduce for Machine Learning Applications. *Proceedings of the Machine Learning on HPC Environments*. New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3146347.3146350>
- Lin, Y., Han, S., Mao, H., Wang, Y., & Dally, W. J. (2018). Deep Gradient Compression: Reducing the Communication Bandwidth for Distributed Training. *International Conference on Learning Representations*. Recuperado de <https://arxiv.org/abs/1712.01887>
- Mathieu, M., Henaff, M., & LeCun, Y. (2013). *Fast Training of Convolutional Networks through FFTs*. <https://doi.org/10.48550/arXiv.1312.5851>

- McMahan, B., Moore, E., Ramage, D., Hampson, S., & Arcas, B. A. y. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *Artificial Intelligence and Statistics*, 1273-1282. PMLR.
- Niu, F., Recht, B., Ré, C., & Wright, S. J. (2011). Hogwild!: A Lock-Free Approach to Parallelizing Stochastic Gradient Descent. *Advances in Neural Information Processing Systems 24*. Recuperado de <https://arxiv.org/abs/1106.5730>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Recuperado de <https://arxiv.org/abs/1912.01703>
- Patarasuk, P., & Yuan, X. (2009). Bandwidth Optimal All-reduce Algorithms for Clusters of Workstations. *Journal of Parallel and Distributed Computing*, 69(2), 117-124. <https://doi.org/10.1016/j.jpdc.2008.09.002>
- Prechelt, L. (1998). Early Stopping — But When? En *Neural Networks: Tricks of the Trade* (pp. 55-69). Springer. https://doi.org/10.1007/3-540-49430-8_3
- Provatas, N., Chalas, I., Konstantinou, I., & Koziris, N. (2025). Strategy-Switch: From All-Reduce to Parameter Server for Faster Efficient Training. *IEEE Access, PP*, 1-1. <https://doi.org/10.1109/ACCESS.2025.3528248>
- Sergeev, A., & Del Balso, M. (2018). Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv preprint arXiv:1802.05799*. Recuperado de <https://arxiv.org/abs/1802.05799>
- Sutskever, I., Martens, J., Dahl, G., & Hinton, G. (2013). On the Importance of Initialization and Momentum in Deep Learning. *Proceedings of the 30th International Conference on Machine Learning*, 1139-1147.
- TensorFlow Developers. (2021). *TensorFlow*. Zenodo. <https://doi.org/10.5281/zenodo.4758419>

Capítulo 8

Anexos

8.1 Anexo A: Repositorio del proyecto

El código fuente, su documentación técnica y el historial completo del desarrollo están alojados en el siguiente repositorio público:

`https://github.com/lminervino18/oxidized-neural-orchestra`

Está organizado como un *workspace* de Cargo, la herramienta de construcción de Rust, y se divide en los siguientes módulos:

- `machine_learning`: el motor de redes neuronales, con las capas, los optimizadores, las funciones de pérdida y la representación de tensores.
- `comms`: la capa de comunicación entre nodos y su protocolo propio.
- `parameter_server`: la implementación de *Parameter Server*.
- `worker`: el runtime del nodo trabajador, que incluye la implementación en anillo de *All-Reduce*.
- `orchestrator`: el plano de control, que asigna los roles y dirige la ejecución de un entrenamiento.
- `node`: el proceso de cómputo, agnóstico del rol que le sea asignado.
- `orchestra-py`: la biblioteca de Python, construida sobre la interfaz de funciones externas.
- `orchestui`: la interfaz interactiva de terminal.
- `benchmarks`: la suite de experimentación y los generadores de gráficos.
- `docker`: los scripts que levantan los entornos de simulación multinodo.
- `docs`: la propuesta, el presente informe y la documentación de diseño del sistema.

8.2 Anexo B: Documentación técnica

Dentro del repositorio se encuentran los siguientes documentos, referenciados a lo largo de este informe:

- `docs/config-schema.md`: la referencia canónica del esquema de configuración de un entrenamiento, y única fuente de verdad del formato que aceptan las interfaces del sistema.
- `docs/architecture-draft.md`: el borrador de arquitectura que sostuvo las discusiones de diseño de las etapas iniciales.

- `docs/report/roadmap.md`: la reconstrucción del recorrido conceptual del proyecto a partir del historial del repositorio, con las decisiones de arquitectura, los debates que las motivaron y su fundamentación bibliográfica. Es el documento de apoyo del que derivan *Metodología aplicada*, *Cronograma* y *Riesgos materializados y lecciones aprendidas*.
- `benchmarks/README.md`: la documentación de la suite de experimentación, con las configuraciones de cada campaña y las instrucciones para reproducirlas.
- Los `README.md` de cada crate, con la documentación de sus abstracciones principales.

8.3 Anexo C: Reproducibilidad de los experimentos

La campaña de benchmarks del sistema es reproducible a partir del material versionado en el repositorio. Se ejecuta mediante `benchmarks/run_issue_benchmarks.py`, cubre 38 configuraciones con tres repeticiones en las suites de throughput y de escalabilidad, que se reportan en media y desvío, y conserva las configuraciones que generaron cada corrida junto con los resultados crudos, los procesados y los generadores de figuras.

El estudio experimental fue desarrollado originalmente como una de las dos monografías individuales que dos de los integrantes del equipo desarrollaron en el marco de la materia Aprendizaje Profundo, sobre un arnés de experimentación propio que no forma parte de este repositorio. Sus configuraciones, sus resultados y el criterio de comparación de cada uno se documentan en *Experimentación y validación*, y las monografías completas se ponen a disposición como documentos separados.

Las semillas de inicialización de los parámetros están fijadas en todas las configuraciones. Debe tenerse presente, no obstante, que las mediciones de tiempo dependen del hardware y de la carga de la máquina en la que se ejecuten, de modo que la reproducibilidad exacta alcanza a las métricas de convergencia y de exactitud, y no a las de rendimiento.

8.4 Cronograma de las actividades realizadas

El plan de actividades de la propuesta fijó dieciséis tareas repartidas en cuatro hitos sobre dos cuatrimestres, con una carga estimada de 1500 horas de equipo. Esta sección contrasta ese plan con la ejecución real, que se extendió entre agosto de 2025 y julio de 2026.

8.4.1 Fases efectivamente recorridas

El desarrollo puede reconstruirse en doce fases, numeradas de 0 a 11. Las fechas son aproximadas y corresponden a rangos de actividad, ya que varias fases se solaparon.

Cuadro 8.1: Fases del desarrollo efectivamente recorridas.

Fase	Período	Contenido
0	Ago–Nov 2025	Propuesta, relevamiento bibliográfico y un primer diseño descartado
1	Nov 2025–Ene 2026	Capa de comunicación
2	Dic 2025–Feb 2026	Parameter Server y Worker
3	Ene–Mar 2026	Motor de redes neuronales
4	Feb–Mar 2026	Orchestrator, interfaz de terminal e interfaz de Python
5	Mar 2026	Multi-servidor, conjuntos de datos y eficiencia de red
6	Mar–May 2026	All-Reduce en anillo y unificación del nodo
7	Abr–May 2026	Early stopping y la saga de bloqueos
8	May–Jun 2026	Strategy Switch, topología y benchmarks
9	Jun 2026	Endurecimiento y exposición de funcionalidad
10	Jun–Jul 2026	Consolidación del benchmark y auditoría de corrección
11	Jul 2026	Optimización final y documentación

8.4.2 Contraste entre lo planificado y lo ejecutado

Lo que se cumplió según lo previsto. El núcleo del plan se completó: el sistema base parametrizable en Rust con su biblioteca de redes neuronales propia (tarea 4), los tres algoritmos (tareas 5 a 7), las optimizaciones de comunicación y sincronización (tareas 8 y 9), las dos interfaces (tarea 10), las pruebas (tarea 11), la simulación sobre distintas configuraciones de nodos (tarea 12) y el análisis de resultados con sus gráficos (tarea 14). El desvío global fue de noventa horas sobre las mil quinientas estimadas, apenas un seis por ciento, pero ese saldo casi neutro esconde compensaciones grandes entre tareas: 550 horas de exceso en unas se cancelaron contra 460 de defecto en otras.

Lo que llevó más tiempo del estimado. El desvío dominante es la tarea 4, que pasó de 150 a 400 horas y explica por sí sola casi la mitad del exceso. Concentra la construcción del sistema base y, sobre todo, la del motor de redes neuronales: la derivación manual del algoritmo de *backpropagation* y la implementación de las capas convolucionales insumieron alrededor de dos meses de depuración de gradientes, muy por encima de lo previsto para un componente que la propuesta trataba como parte de una tarea mayor. El segundo desvío es el Parameter Server (tarea 5), que duplicó su estimación porque llegó a su diseño definitivo después de tres intentos cerrados. El resto del exceso se reparte de forma pareja entre cuatro tareas de acompañamiento que el plan subestimó por igual, las interfaces

(tarea 10), las pruebas (tarea 11), la simulación (tarea 12) y la documentación (tarea 15), cada una del doble o del doble y medio de lo previsto.

Lo que llevó menos. All-Reduce y Strategy-Switch (tareas 6 y 7) costaron la mitad de lo estimado, y la razón es que llegaron sobre infraestructura ya construida: el trabajador, la capa de comunicación y el plano de control existían desde la segunda fase, de modo que lo que quedaba era el algoritmo y no el andamiaje. Las optimizaciones de comunicación y de sincronización (tareas 8 y 9) también quedaron por debajo, aun habiendo absorbido la saga de bloqueos de coordinación de la fase 7, que no figuraba en el plan bajo ninguna tarea. Conviene señalar además que la fase 0 se extendió por cuatro meses de calendario, fue un período dedicado a documentación, relevamiento y trámites; con un primer diseño de arquitectura descartado por completo.

Lo que no se ejecutó. La tarea 13, optimización de carga de cómputo en configuraciones heterogéneas, con 150 horas estimadas, no se abordó.

Lo que se ejecutó sin estar planificado. Aparecieron cuatro líneas no previstas: la selección de topología mediante estadísticas de latencia, la auditoría de corrección distribuida que surgió de la construcción de la suite de benchmarks, la optimización del motor de convolución, y las dos monografías que constituyen los estudios experimentales de este informe, que excedieron en profundidad lo que la tarea 14 contemplaba.

Una observación sobre la estimación. La distribución de horas de la propuesta asignaba 200 horas a investigar las implementaciones distribuidas de TensorFlow y PyTorch (tareas 2 y 3). En la práctica ese estudio se realizó de forma mucho más acotada y en modalidad de consulta puntual, y el esfuerzo se redistribuyó hacia la implementación y la depuración del motor de aprendizaje profundo. La lección de estimación es la habitual y no por conocida menos real: el esfuerzo se subestima sistemáticamente en aquello que no se conoce de antemano, que en este trabajo fue todo lo relativo a la corrección numérica del entrenamiento y a la coordinación distribuida.

8.4.3 Carga horaria efectiva

Cuadro 8.2: Contraste entre la carga horaria estimada en la propuesta y la efectivamente dedicada.

Nro.	Tarea	Horas estimadas	Horas reales
1	Leer y analizar los trabajos previos	100	60
2	Investigar la implementación distribuida de TensorFlow	50	5
3	Investigar la implementación distribuida de PyTorch	50	25
4	Desarrollar el motor de aprendizaje profundo	150	400
5	Implementar Parameter Server	100	200
6	Implementar All-Reduce	100	50
7	Implementar Strategy-Switch	100	50
8	Optimizaciones de comunicación entre nodos	150	100
9	Optimizaciones de sincronización de las copias del modelo	150	100

Nro.	Tarea	Horas estimadas	Horas reales
10	Interfaz de funciones externas en Python e interfaz de terminal	50	100
11	Pruebas unitarias y de integración	50	100
12	Simulación sobre distintas configuraciones de nodos	100	150
13	Optimizaciones de carga en configuraciones heterogéneas	150	0 (no ejecutada)
14	Análisis de resultados y gráficos	100	100
15	Documentación del código y del proceso	50	100
16	Informe final	50	50
	Total	1500	1590

La distribución de esas horas entre los tres integrantes se detalla en *Aportes individuales*.

8.5 Riesgos materializados y lecciones aprendidas

La propuesta identificó tres riesgos iniciales. Los tres se materializaron, uno con una forma distinta de la prevista, y aparecieron otros no contemplados. Esta sección los recorre con la evidencia del historial del proyecto y cierra con las lecciones que dejaron.

8.5.1 Riesgos previstos que se materializaron

La complejidad de la implementación distribuida en Rust. Fue el riesgo dominante. Las garantías de *fearless concurrency* cumplieron lo que prometen, ya que ninguna condición de carrera sobre memoria compartida llegó a la rama principal, pero desplazaron la dificultad hacia la coordinación distribuida, que el compilador no puede verificar porque no es un problema de memoria sino de protocolo. La dificultad real fue sincronizar el estado de fase de cada entidad dentro del algoritmo que ejecuta, sobre todo en All-Reduce, donde cada nodo avanza por pasos que dependen de sus vecinos.

La amplitud del alcance y la estimación de tiempos. La mitigación prevista, priorizar un sistema base funcional y optimizar de forma incremental, se aplicó y funcionó. El costo fue que algunas líneas postergadas no llegaron a cerrarse, según se detalla más abajo.

La dependencia de trabajos previos. Se materializó de una forma no anticipada. Lo previsto era que la interpretación de un algoritmo publicado difiriera de lo documentado; lo que ocurrió fue más sutil: los supuestos de los papers no siempre se cumplen en el régimen propio.

8.5.2 Riesgos no previstos

Aplicar un algoritmo sin verificar la validez de sus supuestos. El sistema implementa la actualización de parámetros libre de bloqueos siguiendo a Niu et al. (Niu et al., 2011), y una auditoría tardía mostró que la variante no era utilizable. Rust no permite obtener dos múltiples referencias mutables a la misma memoria por defecto, de modo que implementar la técnica del paper exigió utilizar estrategias para sortear esa limitación. En particular, se usó el patrón de *interior mutability* para alojar los parámetros y compartir su mutabilidad. El síntoma fue un cuelgue silencioso que solo aparecía con varios trabajadores, y las pruebas existentes no lo detectaron porque ejercitaban únicamente la variante con bloqueo.

8.5.3 Desvíos de alcance

La optimización de carga en configuraciones heterogéneas no se abordó. Estaba enunciada en la propuesta y el entorno de contenedores admitía representar nodos de capacidades distintas, de modo que no hubo un impedimento técnico: la tarea se fue postergando frente a otras prioridades y no llegó a abordarse.

Las pruebas de integración de alto nivel fueron reemplazadas. Las que levantaban un clúster completo desde Python se abandonaron en favor de la suite de benchmarks: sin un estándar acordado para las configuraciones que requerían, su mantenimiento superaba la confianza que aportaban.

8.5.4 Lecciones aprendidas

Conviene tener una medición *end-to-end* desde temprano. La suite de benchmarks llegó al final y con otro propósito, el de comparar estrategias, y aun así terminó destapando tres errores de corrección que llevaban meses en el código: All-Reduce sumaba los gradientes en lugar de promediarlos, con lo que la tasa de aprendizaje efectiva escalaba con la cantidad de workers; al recuperar de los servidores los pesos entrenados, el orquestador no revertía el reparto de capas que él mismo había hecho, de modo que el modelo final quedaba con sus capas desordenadas; y esto se observó recién en *Strategy-Switch* cuando al conmutar de estrategia, los fragmentos se asignaban en un orden distinto del que usaba el trabajador para indexarlos. Los tres eran silenciosos: no provocaban fallos, solo degradaban la exactitud, y ninguna prueba unitaria los habría encontrado porque cada componente hacía correctamente lo suyo.

Lo perfecto es enemigo de lo bueno. Varios tiempos de trabajo sobre la implementación de funcionalidades del sistema difirieron mucho con respecto a lo esperado. La razón fue la “búsqueda de la perfección”, una búsqueda cuyo encuentro se vuelve inalcanzable y que es un problema común en proyectos de este estilo. En particular, el desarrollo del motor de redes neuronales ocupó mucho más tiempo del planificado puesto principalmente a intentos de optimizar el tiempo de cómputo. Conviene siempre trabajar en ítems que conformen cortes transversales de funcionalidad del producto, como sugirió Kent Beck: “*make it work, make it right, make it fast*”.